Case Analítico: LogiPrime & RotaSul

Notebook 02: Perfil Operacional das Rotas

Objetivo: Elaborar uma caracterização profunda do perfil operacional das rotas da **LogiPrime** (SP e RJ) em comparação à **RotaSul** (PR, SC e RS), investigando frota, capacidade, rotas, tempo e produtividade.

Este notebook estrutura a análise em **5 pilares técnicos**:

1. **Composição da Frota e Alocação** – Mix de veículos, sazonalidade e rotatividade
2. **Eficiência de Capacidade (Fill Rate)** – Ocupação e gargalos físicos (Weigh-out vs. Cube-out)
3. **Capilaridade e Desenho de Rota** – Extensão, densidade de paradas e dispersão geográfica
4. **Perfil de Tempos e Ritmo Operacional** – Matriz de tempo, velocidade e estouro de jornada
5. **Drop Size e Produtividade** – Tamanho de entrega e eficiência de descarga

```
# Importação das bibliotecas necessárias
#pip install polars pandas matplotlib seaborn
import polars as pl
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

# Configurações visuais para os gráficos
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (14, 7)
plt.rcParams['font.size'] = 10
plt.rcParams['axes.titlesize'] = 12
plt.rcParams['axes.labelsize'] = 11

# Caminho dos dados processados
path_dados_limpos = "../data/03_processed/dados_limpos.parquet"

# Carregando a base de dados consolidada
df = pl.read_parquet(path_dados_limpos)

print(f"Base de dados carregada com sucesso")
print(f"Total de registros: {df.height:,}")
print(f"Total de colunas: {df.width}")
print(f"Período coberto: {df.select('data_rota').min()[0,0]} a
```

```
{df.select('data_rota').max()[0,0]}")
print(f"\nEmpresas: {df.select('empresa').unique().to_pandas()
['empresa'].tolist()}")
print(f"Estados: {df.select('uf').unique().to_pandas()
['uf'].tolist()}")
print(f"Tipos de Veículos:
{df.select('tipo_veiculo').unique().to_pandas()
['tipo_veiculo'].tolist()}")
```

```
Base de dados carregada com sucesso
Total de registros: 3,455,659
Total de colunas: 28
Período coberto: 2024-01-01 a 2024-12-31

Empresas: ['LogiPrime', 'RotaSul']
Estados: ['PR', 'SP', 'RJ', 'SC', 'RS']
Tipos de Veículos: ['CARRETA', 'TOCO', 'VUC', 'TRUCK']
```

1. COMPOSIÇÃO DA FROTA E ALOCAÇÃO

1.1 Mix de Veículos por Empresa

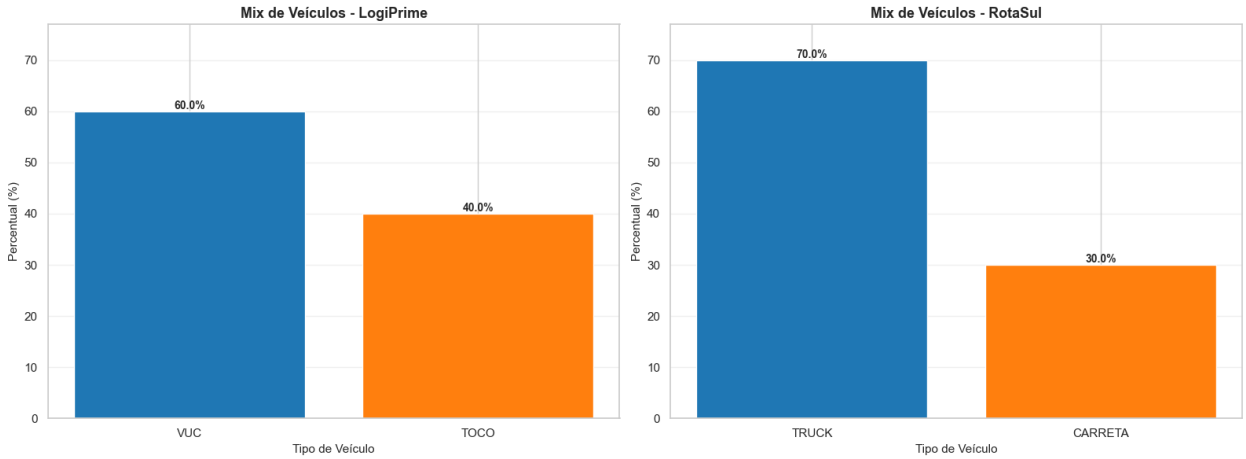
A escolha do veículo determina a dinâmica de custo e capacidade. LogiPrime, focada em centros urbanos, deve privilegiar VUCs e TOCOs. RotaSul, com capilaridade regional, deve depender de TRUCKs e CARRETAs para longas distâncias.

```
# 1.1 Mix de Veículos por Empresa
mix_veiculo_empresa = df.group_by(['empresa', 'tipo_veiculo']).agg(
    pl.len().alias('qtd_registros')
).with_columns([
    (pl.col('qtd_registros') /
pl.col('qtd_registros').sum().over('empresa') *
100).alias('percentual')
]).sort('empresa', 'qtd_registros', descending=[False, True])

# Convertendo para Pandas para visualização
mix_df = mix_veiculo_empresa.to_pandas()

print(" DISTRIBUIÇÃO DO MIX DE VEÍCULOS POR EMPRESA")
print("="*70)
for empresa in mix_df['empresa'].unique():
    empresa_data = mix_df[mix_df['empresa'] == empresa]
    print(f"\n{empresa}")
    print("-"*70)
    for _, row in empresa_data.iterrows():
        print(f"  {row['tipo_veiculo']:10s}
{row['qtd_registros']:>8,d} registros  ({row['percentual']:>5.1f}%)")

# Visualização 1: Gráfico de barras comparativo
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for idx, empresa in enumerate(mix_df['empresa'].unique()):
    empresa_data = mix_df[mix_df['empresa'] ==
empresa].sort_values('qtd_registros', ascending=False)

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']
    ax = axes[idx]
    bars = ax.bar(empresa_data['tipo_veiculo'],
empresa_data['percentual'], color=colors[:len(empresa_data)])

    ax.set_title(f'Mix de Veículos - {empresa}', fontsize=13,
fontweight='bold')
    ax.set_ylabel('Percentual (%)', fontsize=11)
    ax.set_xlabel('Tipo de Veículo', fontsize=11)
    ax.set_ylim(0, max(mix_df['percentual']) * 1.1)

    # Adicionando valores nas barras
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.1f}%', ha='center', va='bottom',
fontsize=10, fontweight='bold')

    ax.grid(axis='y', alpha=0.3)
    ax.set_axisbelow(True)

plt.tight_layout()
plt.show()

# Tabela resumida
print("\n TABELA RESUMIDA")
print("="*70)
summary_table = mix_df.pivot_table(
    values='percentual',
    index='tipo_veiculo',
    columns='empresa',
    aggfunc='sum',
    fill_value=0
)
print(summary_table.round(1))
```

```
 DISTRIBUIÇÃO DO MIX DE VEÍCULOS POR EMPRESA
======================================================================

LogiPrime
----------------------------------------------------------------------
  VUC        1,811,207 registros  ( 60.0%)
  TOCO       1,205,664 registros  ( 40.0%)
```

```
RotaSul
----------------------------------------------------------------------
  TRUCK       307,014 registros  ( 70.0%)
  CARRETA     131,774 registros  ( 30.0%)
```




```
 TABELA RESUMIDA
======================================================================
empresa       LogiPrime  RotaSul
tipo_veiculo
CARRETA             0.0     30.0
TOCO               40.0      0.0
TRUCK               0.0     70.0
VUC                60.0      0.0
```

- A análise do mix de veículos presentes na frota de cada empresa demostra que as duas empresas operam sobre diferentes e ligadas a localidade onde cada uma opera. A empresa LogiPrime opera 100% com veículos de pequeno e médio porte (60% VUC e 40% TOCO), o que esta diretamente relacionada às restrições de circulação em grandes centros urbanos como São Paulo e Rio de Janeiro, que são cidades com zonas de restrição a veículos pesados e malha viária congestionada que inviabilizam o uso de carretas ou trucks. Já a empresa RotaSul, opera com a sua frota composta por veículos pesados (70% TRUCK e 30% CARRETA) que sejam capazes de compor grandes cargas e percorrer longas distâncias entre municípios do Sul do Brasil.

1.2 Proporção por Estado e Região

Analisar se os padrões de mix variam geograficamente dentro da mesma empresa. SP e RJ se comportam da mesma forma? PR, SC e RS possuem o mesmo mix?

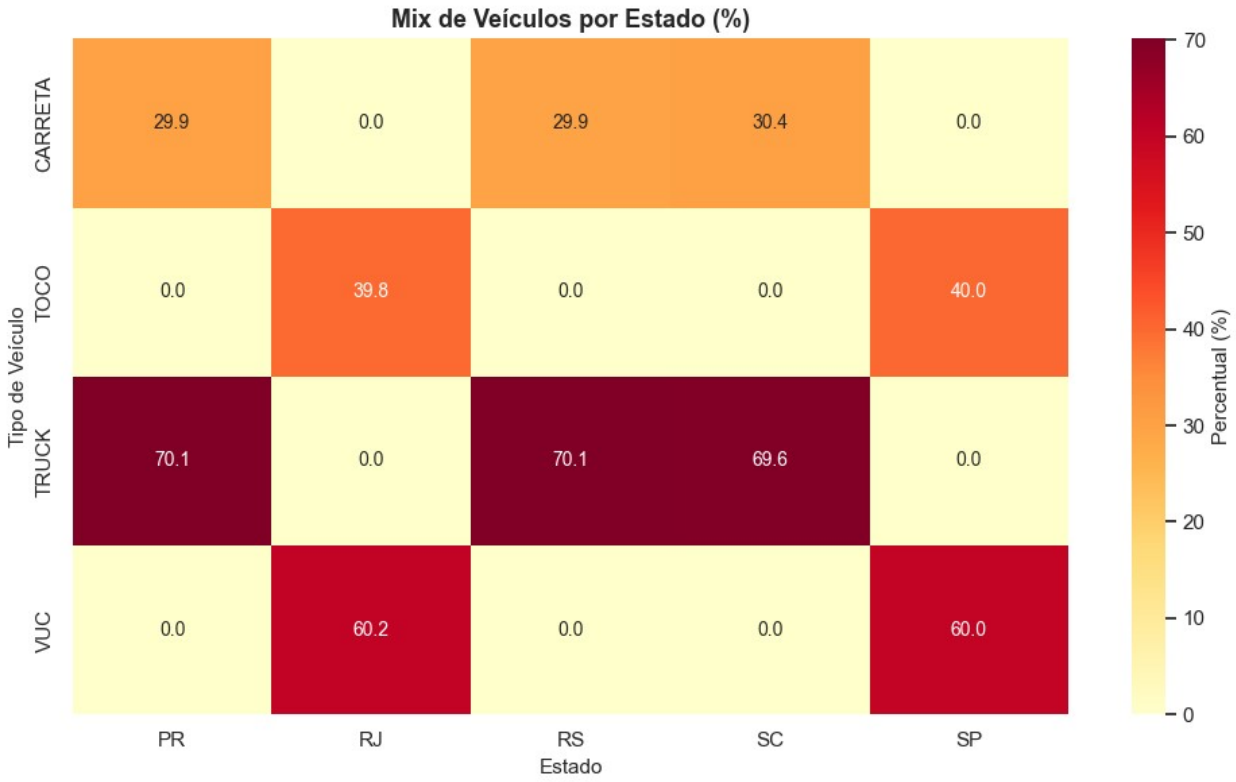
```
# 1.2 Mix de Veículos por Estado
mix_veiculo_estado = df.group_by(['uf', 'tipo_veiculo']).agg(
    pl.len().alias('qtd_registros')
).with_columns([
```

```
    (pl.col('qtd_registros') /
pl.col('qtd_registros').sum().over('uf') * 100).alias('percentual')
]).sort('uf', 'qtd_registros', descending=[False, True])

mix_estado_df = mix_veiculo_estado.to_pandas()

print("\n DISTRIBUIÇÃO DO MIX DE VEÍCULOS POR ESTADO")
print("="*70)
for uf in mix_estado_df['uf'].unique():
    uf_data = mix_estado_df[mix_estado_df['uf'] == uf]
    print(f"\n{uf}")
    print("-"*70)
    for _, row in uf_data.iterrows():
        print(f"  {row['tipo_veiculo']:10s}
{row['qtd_registros']:>8,d} registros  ({row['percentual']:>5.1f}%)")
```

```
 DISTRIBUIÇÃO DO MIX DE VEÍCULOS POR ESTADO
======================================================================

PR
----------------------------------------------------------------------
  TRUCK       123,251 registros  ( 70.1%)
  CARRETA      52,516 registros  ( 29.9%)

RJ
----------------------------------------------------------------------
  VUC         543,074 registros  ( 60.2%)
  TOCO        358,619 registros  ( 39.8%)

RS
----------------------------------------------------------------------
  TRUCK        91,395 registros  ( 70.1%)
  CARRETA      38,904 registros  ( 29.9%)

SC
----------------------------------------------------------------------
  TRUCK        92,368 registros  ( 69.6%)
  CARRETA      40,354 registros  ( 30.4%)

SP
----------------------------------------------------------------------
  VUC        1,268,133 registros  ( 60.0%)
  TOCO        847,045 registros  ( 40.0%)
```

```
# Heatmap de Mix por Estado e Veículo
pivot_mix = mix_estado_df.pivot_table(
    values='percentual',
    index='tipo_veiculo',
    columns='uf',
```

```
    fill_value=0
)

fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(pivot_mix, annot=True, fmt='.1f', cmap='YlOrRd',
cbar_kws={'label': 'Percentual (%)'}, ax=ax)
ax.set_title('Mix de Veículos por Estado (%)', fontsize=13,
fontweight='bold')
ax.set_xlabel('Estado', fontsize=11)
ax.set_ylabel('Tipo de Veículo', fontsize=11)
plt.tight_layout()
plt.show()
```



- A distribuição por estado reforça que essa segmentação é um reflexo da demanda de cada mercado ao qual cada empresa opera. Os estados de SP e RJ concentram exclusivamente VUCs e TOCOs, enquanto PR, SC e RS registram apenas TRUCKs e CARRETAs, sem cruzamento entre os perfis. Isso tem implicação direta para qualquer estratégia de integração de frota pós-fusão, pois as operações são geograficamente complementares, mas operacionalmente incompatíveis em termos de ativos físicos.

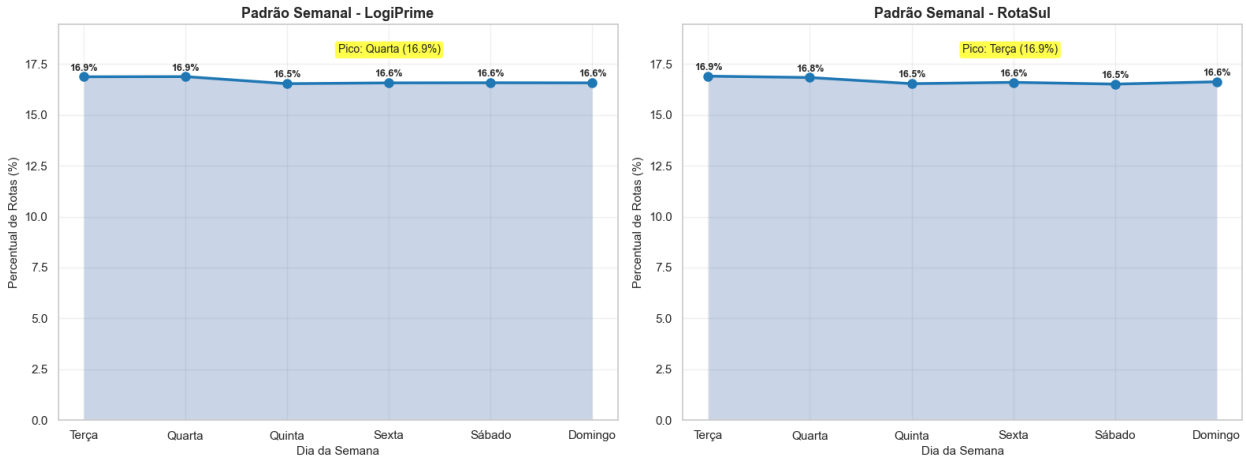
1.3 Sazonalidade e Padrões Semanais

Verificar se há concentração de viagens em dias específicos da semana. A RotaSul concentra suas rotas longas às segundas-feiras? A LogiPrime tem picos às sextas?

```
# 1.3 Sazonalidade e Padrões Semanais
# Preparando dados com dia da semana
df_sazonalidade = df.with_columns([
    pl.col('data_rota').dt.weekday().alias('dia_semana_num'),
    pl.col('data_rota').dt.strftime('%A').alias('dia_semana')
]).group_by(['empresa', 'dia_semana_num']).agg([
    pl.len().alias('qtd_rotas')
]).with_columns([
    (pl.col('qtd_rotas') / pl.col('qtd_rotas').sum().over('empresa') *
100).alias('percentual')
]).sort('empresa', 'dia_semana_num')

# Mapeando números para nomes dos dias
dia_map = {0: 'Segunda', 1: 'Terça', 2: 'Quarta', 3: 'Quinta', 4:
'Sexta', 5: 'Sábado', 6: 'Domingo'}
sazon_df = df_sazonalidade.to_pandas()
sazon_df['dia_semana_nome'] = sazon_df['dia_semana_num'].map(dia_map)

print("\n DISTRIBUIÇÃO DE ROTAS POR DIA DA SEMANA")
print("="*70)
for empresa in sazon_df['empresa'].unique():
    empresa_data = sazon_df[sazon_df['empresa'] ==
empresa].sort_values('dia_semana_num')
    print(f"\n{empresa}")
    print("-"*70)
    for _, row in empresa_data.iterrows():
        print(f"  {row['dia_semana_nome']:10s} {row['qtd_rotas']:>8,d}
rotas  ({row['percentual']:>5.1f}%)")

# Visualização: Padrão semanal por empresa
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for idx, empresa in enumerate(sazon_df['empresa'].unique()):
    empresa_data = sazon_df[sazon_df['empresa'] ==
empresa].sort_values('dia_semana_num')

    ax = axes[idx]
    linha = ax.plot(empresa_data['dia_semana_nome'],
empresa_data['percentual'],
                    marker='o', linewidth=2.5, markersize=8,
color='#1f77b4')
    ax.fill_between(range(len(empresa_data)),
empresa_data['percentual'], alpha=0.3)

    ax.set_title(f'Padrão Semanal - {empresa}', fontsize=13,
fontweight='bold')
    ax.set_ylabel('Percentual de Rotas (%)', fontsize=11)
    ax.set_xlabel('Dia da Semana', fontsize=11)
    ax.set_ylim(0, max(sazon_df['percentual']) * 1.15)
    ax.grid(True, alpha=0.3)
```

```
    # Adicionando valores nos pontos
    for x, y in zip(range(len(empresa_data)),
empresa_data['percentual']):
        ax.text(x, y + 0.3, f'{y:.1f}%', ha='center', fontsize=9,
fontweight='bold')

    # Identificando picos
    pico_idx = empresa_data['percentual'].idxmax()
    pico_dia = empresa_data.loc[pico_idx, 'dia_semana_nome']
    pico_pct = empresa_data.loc[pico_idx, 'percentual']
    ax.text(0.5, 0.95, f'Pico: {pico_dia} ({pico_pct:.1f}%)',
            transform=ax.transAxes, fontsize=10,
verticalalignment='top',
            bbox=dict(boxstyle='round', facecolor='yellow',
alpha=0.7))

plt.tight_layout()
plt.show()
```

```
 DISTRIBUIÇÃO DE ROTAS POR DIA DA SEMANA
======================================================================

LogiPrime
----------------------------------------------------------------------
  Terça       508,978 rotas  ( 16.9%)
  Quarta      509,212 rotas  ( 16.9%)
  Quinta      498,736 rotas  ( 16.5%)
  Sexta       499,909 rotas  ( 16.6%)
  Sábado      500,101 rotas  ( 16.6%)
  Domingo     499,935 rotas  ( 16.6%)

RotaSul
----------------------------------------------------------------------
  Terça        74,162 rotas  ( 16.9%)
  Quarta       73,863 rotas  ( 16.8%)
  Quinta       72,543 rotas  ( 16.5%)
  Sexta        72,823 rotas  ( 16.6%)
  Sábado       72,451 rotas  ( 16.5%)
  Domingo      72,946 rotas  ( 16.6%)
```



- Quanto à sazonalidade semanal, ambas as empresas apresentam distribuição relativamente uniforme ao longo dos dias úteis, sem concentrações expressivas. A empresa LogiPrime tem leve pico na quarta-feira (16,9%) e a RotaSul na terça-feira (16,9%). Isso indica que nenhuma das duas possui desequilíbrio crítico de demanda ao longo da semana, o que simplifica o planejamento de escala de motoristas e veículos.

1.4 Produtividade por Placa (Giro da Frota)

Contar quantas rotas únicas uma mesma placa realiza no mês. Na operação urbana da LogiPrime, é possível que um mesmo VUC faça mais de uma rota no mesmo dia (turnos diferentes), enquanto na RotaSul o veículo pode ficar dias na mesma rota.

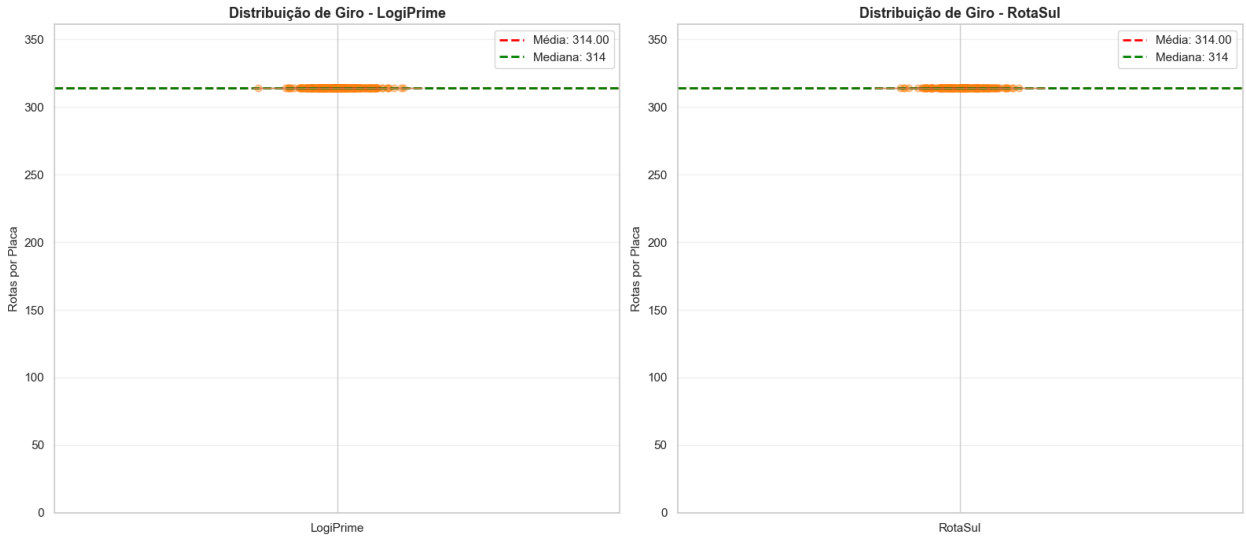
```
# 1.4 Produtividade por Placa (Giro da Frota)
giro_frota = df.group_by(['empresa', 'placa_veiculo']).agg([
    pl.col('codigo_rota').n_unique().alias('rotas_por_placa'),
    pl.col('tipo_veiculo').first().alias('tipo_veiculo'),
    pl.col('uf').first().alias('uf')
]).with_columns([

pl.col('rotas_por_placa').mean().over('empresa').alias('media_rotas_em
presa'),

pl.col('rotas_por_placa').median().over('empresa').alias('mediana_rota
s_empresa')
]).sort('empresa', 'rotas_por_placa', descending=[False, True])

giro_df = giro_frota.to_pandas()

# Estatísticas por empresa
print("\n GIRO DA FROTA (ROTAS POR PLACA)")
print("="*70)
for empresa in giro_df['empresa'].unique():
    empresa_data = giro_df[giro_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*70)
```

```
    print(f"  Total de placas: {len(empresa_data):,}")
    print(f"  Média de rotas por placa:
{empresa_data['rotas_por_placa'].mean():.2f}")
    print(f"  Mediana de rotas por placa:
{empresa_data['rotas_por_placa'].median():.0f}")
    print(f"  Mínimo: {empresa_data['rotas_por_placa'].min():.0f} |
Máximo: {empresa_data['rotas_por_placa'].max():.0f}")

    # Top 5 placas mais produtivas
    print(f"\n  Top 5 Placas Mais Produtivas:")
    top5 = empresa_data.nlargest(5, 'rotas_por_placa')
[['placa_veiculo', 'tipo_veiculo', 'rotas_por_placa']]
    for _, row in top5.iterrows():
        print(f"    {row['placa_veiculo']} ({row['tipo_veiculo']:8s})
- {row['rotas_por_placa']:.0f} rotas")
```

```
 GIRO DA FROTA (ROTAS POR PLACA)
======================================================================

LogiPrime
----------------------------------------------------------------------
  Total de placas: 300
  Média de rotas por placa: 314.00
  Mediana de rotas por placa: 314
  Mínimo: 314 | Máximo: 314

  Top 5 Placas Mais Produtivas:
    A-TOCO-00052 (TOCO    ) - 314 rotas
    A-VUC-00091 (VUC     ) - 314 rotas
    A-TOCO-00007 (TOCO    ) - 314 rotas
    A-TOCO-00038 (TOCO    ) - 314 rotas
    A-VUC-00074 (VUC     ) - 314 rotas

RotaSul
----------------------------------------------------------------------
  Total de placas: 200
  Média de rotas por placa: 314.00
  Mediana de rotas por placa: 314
  Mínimo: 314 | Máximo: 314

  Top 5 Placas Mais Produtivas:
    B-CAR-00046 (CARRETA ) - 314 rotas
    B-TRK-00024 (TRUCK   ) - 314 rotas
    B-CAR-00023 (CARRETA ) - 314 rotas
    B-CAR-00013 (CARRETA ) - 314 rotas
    B-CAR-00007 (CARRETA ) - 314 rotas
```

```
# Visualização: Distribuição de giro por empresa
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
```

```
for idx, empresa in enumerate(giro_df['empresa'].unique()):
    empresa_data = giro_df[giro_df['empresa'] == empresa]

    ax = axes[idx]

    # Boxplot com distribuição
    bp = ax.boxplot([empresa_data['rotas_por_placa']], vert=True,
patch_artist=True,
                     labels=[empresa], widths=0.3)

    # Colorindo o boxplot
    for patch in bp['boxes']:
        patch.set_facecolor('#1f77b4')
        patch.set_alpha(0.7)

    # Adicionando scatter dos pontos
    y = empresa_data['rotas_por_placa']
    x = np.random.normal(1, 0.04, len(y))
    ax.scatter(x, y, alpha=0.4, s=40, color='#ff7f0e')

    # Adicionando linhas de média e mediana
    media = empresa_data['rotas_por_placa'].mean()
    mediana = empresa_data['rotas_por_placa'].median()
    ax.axhline(media, color='red', linestyle='--', linewidth=2,
label=f'Média: {media:.2f}')
    ax.axhline(mediana, color='green', linestyle='--', linewidth=2,
label=f'Mediana: {mediana:.0f}')

    ax.set_title(f'Distribuição de Giro - {empresa}', fontsize=13,
fontweight='bold')
    ax.set_ylabel('Rotas por Placa', fontsize=11)
    ax.legend(loc='upper right')
    ax.grid(axis='y', alpha=0.3)
    ax.set_ylim(0, empresa_data['rotas_por_placa'].quantile(0.95) *
1.15)

plt.tight_layout()
plt.show()
```




```python
# Análise por tipo de veículo
print("\n GIRO MÉDIO POR TIPO DE VEÍCULO")
print("="*70)
giro_veiculo = giro_df.groupby(['empresa', 'tipo_veiculo'])
['rotas_por_placa'].agg(['mean', 'median', 'count']).round(2)
print(giro_veiculo)
```

```
 GIRO MÉDIO POR TIPO DE VEÍCULO
======================================================================
                          mean  median  count
empresa   tipo_veiculo
LogiPrime TOCO           314.0   314.0    120
          VUC            314.0   314.0    180
RotaSul   CARRETA        314.0   314.0     60
          TRUCK          314.0   314.0    140
```

- O indicador de giro de frota (rotas por placa) apresentou um dado interessante, pois a média e mediana idênticas de 314 rotas por placa ao longo do ano para todas as categorias de veículo, em ambas as empresas. Essa uniformidade artificial sugere que o dado pode ter sido gerado sinteticamente ou que a metodologia de construção do dataset padronizou o giro. Na prática, poderia ser esperado que VUCs da LogiPrime realizassem múltiplas rotas diárias (turnos) enquanto TRUCKs e CARRETAs da RotaSul permanecessem dias em uma única rota de longa distância, o que seria uma distinção que teria implicações importantes para o dimensionamento de frota pós-fusão.

---

2. EFICIÊNCIA DE CAPACIDADE (FILL RATE)

Um caminhão rodando vazio é custo afundado. Aqui analisamos a ocupação de peso e volume contra os limites técnicos de cada veículo, identificando gargalos e padrões de utilização.

2.1 Ocupação de Peso por Tipo de Veículo

Comparar a média e mediana do `peso_kg_total_rota` contra o limite de peso de cada categoria.

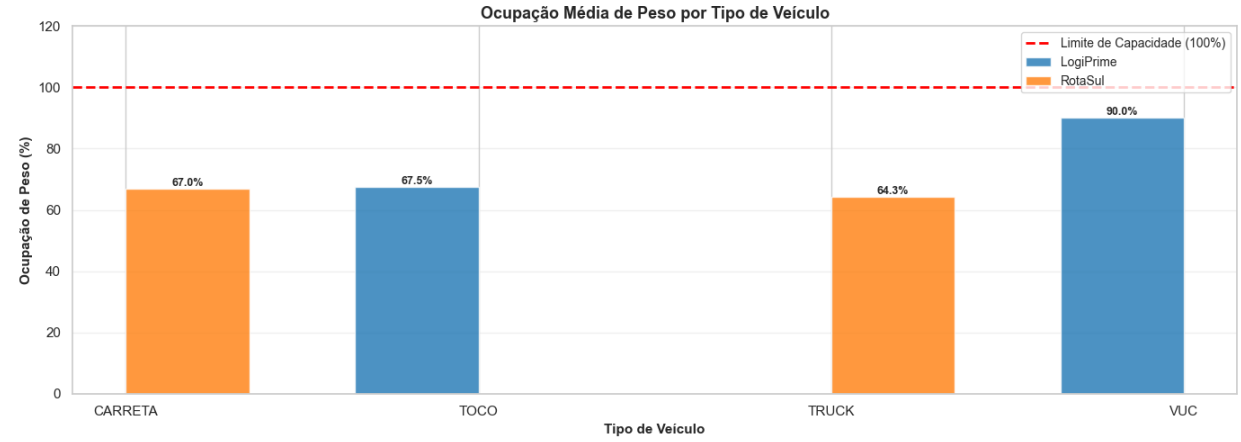
```
# 2.1 Ocupação de Peso por Tipo de Veículo
# Limites de capacidade em kg
capacidades_peso = {
    "VUC": 3000,
    "VAN": 1500,
    "TOCO": 8000,
    "TRUCK": 14000,
    "CARRETA": 28000
}

# Análise de ocupação de peso
ocupacao_peso = df.group_by(['empresa', 'tipo_veiculo']).agg([
    pl.col('peso_kg_total_rota').mean().alias('media_peso'),
    pl.col('peso_kg_total_rota').median().alias('mediana_peso'),
    pl.col('peso_kg_total_rota').min().alias('min_peso'),
    pl.col('peso_kg_total_rota').max().alias('max_peso'),
    pl.col('peso_kg_total_rota').std().alias('std_peso'),
    pl.len().alias('qtd_rotas')
]).to_pandas()

# Adicionando capacidade máxima e percentual de ocupação
ocupacao_peso['capacidade_max'] =
ocupacao_peso['tipo_veiculo'].map(capacidades_peso)
ocupacao_peso['ocupacao_media_%'] = (ocupacao_peso['media_peso'] /
ocupacao_peso['capacidade_max']) * 100
ocupacao_peso['ocupacao_mediana_%'] = (ocupacao_peso['mediana_peso'] /
ocupacao_peso['capacidade_max']) * 100

# Adicionando coluna de percentual disponível
ocupacao_peso['disponivel_%'] = 100 - ocupacao_peso['ocupacao_media_
%']

print("\n OCUPAÇÃO DE PESO POR TIPO DE VEÍCULO")
print("="*100)

for empresa in ocupacao_peso['empresa'].unique():
    empresa_data = ocupacao_peso[ocupacao_peso['empresa'] ==
empresa].sort_values('tipo_veiculo')
    print(f"\n{empresa}")
    print("-"*100)
    print(f"{'Veículo':<10} {'Média (kg)':<12} {'Cap.Max (kg)':<15}
{'Ocupação %':<12} {'Disponível %':<13} {'Mediana':<12} {'Std
Dev':<12}")
    print("-"*100)
```

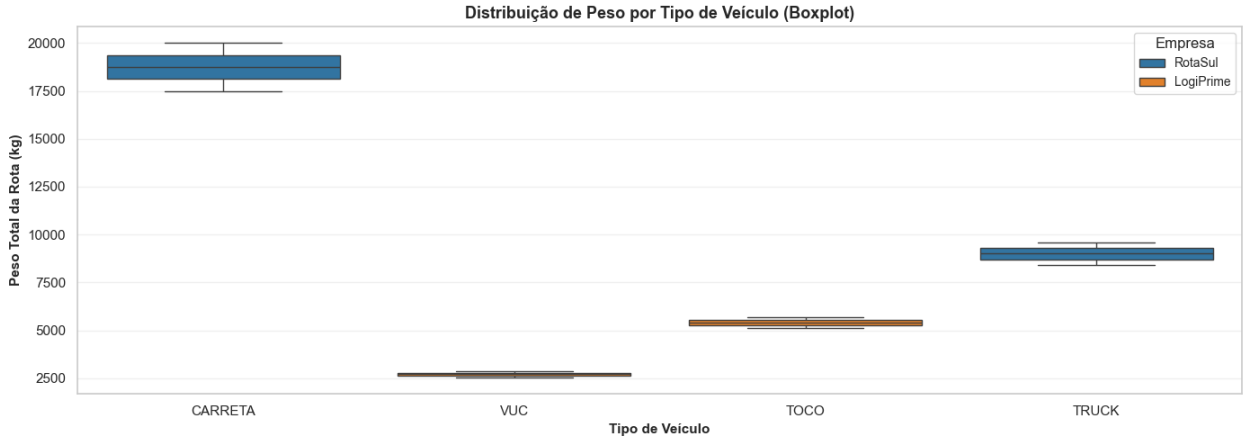
```
    for _, row in empresa_data.iterrows():
        print(f"{row['tipo_veiculo']:<10} {row['media_peso']:>11,.0f}
{row['capacidade_max']:>14,.0f} {row['ocupacao_media_%']:>11.1f}%
{row['disponivel_%']:>12.1f}% {row['mediana_peso']:>11,.0f}
{row['std_peso']:>11,.0f}")
```

```
 OCUPAÇÃO DE PESO POR TIPO DE VEÍCULO
==========================================================================
==============================

LogiPrime
--------------------------------------------------------------------------
------------------------------
Veículo    Média (kg)   Cap.Max (kg)    Ocupação %   Disponível %
Mediana      Std Dev
--------------------------------------------------------------------------
------------------------------
TOCO             5,401          8,000        67.5%         32.5%
5,401         173
VUC              2,700          3,000        90.0%         10.0%
2,700          87

RotaSul
--------------------------------------------------------------------------
------------------------------
Veículo    Média (kg)   Cap.Max (kg)    Ocupação %   Disponível %
Mediana      Std Dev
--------------------------------------------------------------------------
------------------------------
CARRETA         18,752         28,000        67.0%         33.0%
18,757         724
TRUCK            9,001         14,000        64.3%         35.7%
9,000         347
```

```
# Visualização: Ocupação média por tipo de veículo
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Gráfico 1: Ocupação média
ax1 = axes[0]
ocupacao_pivot = ocupacao_peso.pivot_table(
    values='ocupacao_media_%',
    index='tipo_veiculo',
    columns='empresa',
    aggfunc='first'
)

x = np.arange(len(ocupacao_pivot.index))
width = 0.35
```

```
bars1 = ax1.bar(x - width/2, ocupacao_pivot.iloc[:, 0], width,
label=ocupacao_pivot.columns[0], alpha=0.8, color='#1f77b4')
bars2 = ax1.bar(x + width/2, ocupacao_pivot.iloc[:, 1], width,
label=ocupacao_pivot.columns[1], alpha=0.8, color='#ff7f0e')

# Linha de referência 100%
ax1.axhline(100, color='red', linestyle='--', linewidth=2,
label='Limite de Capacidade (100%)')

ax1.set_xlabel('Tipo de Veículo', fontsize=11, fontweight='bold')
ax1.set_ylabel('Ocupação de Peso (%)', fontsize=11, fontweight='bold')
ax1.set_title('Ocupação Média de Peso por Tipo de Veículo',
fontsize=13, fontweight='bold')
ax1.set_xticks(x)
ax1.set_xticklabels(ocupacao_pivot.index)
ax1.legend(fontsize=10)
ax1.grid(axis='y', alpha=0.3)
ax1.set_ylim(0, 120)

# Adicionando valores nas barras
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax1.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.1f}%', ha='center', va='bottom',
fontsize=9, fontweight='bold')

# Gráfico 2: Boxplot de distribuição de peso por empresa
ax2 = axes[1]

df_peso_plot = df.select(['empresa', 'tipo_veiculo',
'peso_kg_total_rota']).to_pandas()
df_peso_plot = df_peso_plot.dropna()

sns.boxplot(data=df_peso_plot, x='tipo_veiculo',
y='peso_kg_total_rota', hue='empresa', ax=ax2, palette=['#1f77b4',
'#ff7f0e'])
ax2.set_xlabel('Tipo de Veículo', fontsize=11, fontweight='bold')
ax2.set_ylabel('Peso Total da Rota (kg)', fontsize=11,
fontweight='bold')
ax2.set_title('Distribuição de Peso por Tipo de Veículo (Boxplot)',
fontsize=13, fontweight='bold')
ax2.legend(title='Empresa', fontsize=10)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```





- A análise de ocupação revela padrões distintos de utilização e aponta gargalos físicos diferentes entre as duas operações. Em peso, a LogiPrime demonstra utilização elevada nos seus VUCs, que operam com ocupação média de 90% da capacidade de peso (2.700 kg dos 3.000 kg permitidos), enquanto os TOCOs ficam em 67,5%. A RotaSul apresenta ocupação de 67% nas CARRETAs e 64,3% nos TRUCKs, que são números menores em termos percentuais, mas que representam cargas absolutas muito maiores por viagem (18.752 kg médios em CARRETAs).

2.2 Ocupação Volumétrica e Identificação de Gargalos

Análise similar ao peso, mas considerando volume (m^3) versus capacidade volumétrica. Identificar se o gargalo é por Peso (Weigh-out) ou por Volume (Cube-out).

```
# 2.2 Ocupação Volumétrica
capacidades_volume = {
    "VUC": 20,
    "VAN": 12,
    "TOCO": 40,
    "TRUCK": 60,
    "CARRETA": 100
}

# Análise de ocupação de volume
```

```python
ocupacao_volume = df.group_by(['empresa', 'tipo_veiculo']).agg([
    pl.col('volume_m3_total_rota').mean().alias('media_volume'),
    pl.col('volume_m3_total_rota').median().alias('mediana_volume'),
    pl.col('volume_m3_total_rota').min().alias('min_volume'),
    pl.col('volume_m3_total_rota').max().alias('max_volume'),
    pl.col('volume_m3_total_rota').std().alias('std_volume'),
]).to_pandas()

# Adicionando capacidade máxima
ocupacao_volume['capacidade_max'] =
ocupacao_volume['tipo_veiculo'].map(capacidades_volume)
ocupacao_volume['ocupacao_media_%'] = (ocupacao_volume['media_volume']
/ ocupacao_volume['capacidade_max']) * 100
ocupacao_volume['ocupacao_mediana_%'] =
(ocupacao_volume['mediana_volume'] /
ocupacao_volume['capacidade_max']) * 100

# Adicionando coluna de percentual disponível
ocupacao_volume['disponivel_%'] = 100 -
ocupacao_volume['ocupacao_media_%']

print("\n OCUPAÇÃO VOLUMÉTRICA POR TIPO DE VEÍCULO")
print("="*100)

for empresa in ocupacao_volume['empresa'].unique():
    empresa_data = ocupacao_volume[ocupacao_volume['empresa'] ==
empresa].sort_values('tipo_veiculo')
    print(f"\n{empresa}")
    print("-"*100)
    print(f"{'Veículo':<10} {'Média (m³)':<12} {'Cap.Max (m³)':<15}
{'Ocupação %':<12} {'Disponível %':<13} {'Mediana':<12} {'Std
Dev':<12}")
    print("-"*100)

    for _, row in empresa_data.iterrows():
        print(f"{row['tipo_veiculo']:<10}
{row['media_volume']:>11,.1f} {row['capacidade_max']:>14,.1f}
{row['ocupacao_media_%']:>11.1f}% {row['disponivel_%']:>12.1f}%
{row['mediana_volume']:>11,.1f} {row['std_volume']:>11,.1f}")
```

```
 OCUPAÇÃO VOLUMÉTRICA POR TIPO DE VEÍCULO
============================================================================
==============================

RotaSul
----------------------------------------------------------------------------
------------------------------
Veículo    Média (m³)   Cap.Max (m³)    Ocupação %   Disponível %
```

```
Mediana      Std Dev
------------------------------------------------------------------------
------------------------------
CARRETA           75.0          100.0        75.0%         25.0%
75.0         2.9
TRUCK             45.0           60.0        75.0%         25.0%
45.0         1.7

LogiPrime
------------------------------------------------------------------------
------------------------------
Veículo    Média (m³)   Cap.Max (m³)    Ocupação %   Disponível %
Mediana      Std Dev
------------------------------------------------------------------------
------------------------------
TOCO              36.0           40.0        90.0%         10.0%
36.0         1.2
VUC               18.0           20.0        90.0%         10.0%
18.0         0.6
```

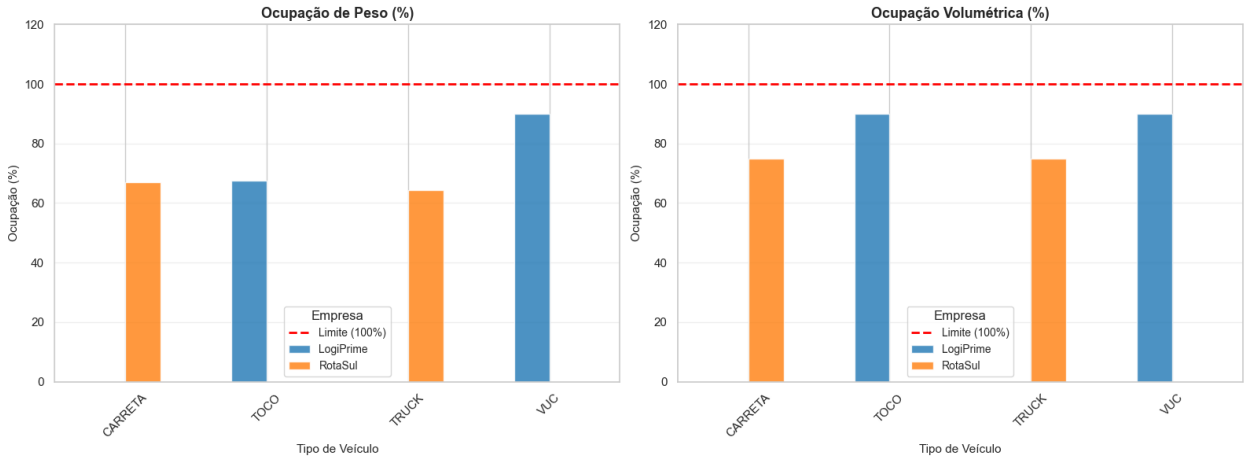
- No eixo volumétrico, o padrão se repete de forma mais expressiva. A empresa LogiPrime opera com 90% de ocupação volumétrica tanto nos VUCs quanto nos TOCOs, enquanto a RotaSul fica em 75% nas CARRETAs e TRUCKs. Essa diferença de ~15 pontos percentuais no fill rate volumétrico indica que a RotaSul possui espaço ocioso relevante nos seus veículos, o que representa uma oportunidade direta de consolidação de cargas e aumento de receita por viagem sem incremento de custos fixos.

2.3 Weigh-out vs. Cube-out: Identificação do Gargalo

Matriz de identificação de qual é o principal limitante: o peso (Weigh-out) ou o volume (Cube-out). Se ambos estão próximos de 100%, o veículo está otimizado. Se um está muito abaixo de 100%, há desperdício.

```
# Visualização: Comparação peso vs volume
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Gráfico 1: Ocupação de Peso
ax1 = axes[0]
peso_pivot = ocupacao_peso.pivot_table(
    values='ocupacao_media_%',
    index='tipo_veiculo',
    columns='empresa'
)
peso_pivot.plot(kind='bar', ax=ax1, alpha=0.8, color=['#1f77b4',
'#ff7f0e'])
ax1.axhline(100, color='red', linestyle='--', linewidth=2,
label='Limite (100%)')
ax1.set_title('Ocupação de Peso (%)', fontsize=13, fontweight='bold')
ax1.set_ylabel('Ocupação (%)', fontsize=11)
```

```
ax1.set_xlabel('Tipo de Veículo', fontsize=11)
ax1.legend(title='Empresa', fontsize=10)
ax1.grid(axis='y', alpha=0.3)
ax1.set_ylim(0, 120)
ax1.set_xticklabels(ax1.get_xticklabels(), rotation=45)

# Gráfico 2: Ocupação de Volume
ax2 = axes[1]
volume_pivot = ocupacao_volume.pivot_table(
    values='ocupacao_media_%',
    index='tipo_veiculo',
    columns='empresa'
)
volume_pivot.plot(kind='bar', ax=ax2, alpha=0.8, color=['#1f77b4',
'#ff7f0e'])
ax2.axhline(100, color='red', linestyle='--', linewidth=2,
label='Limite (100%)')
ax2.set_title('Ocupação Volumétrica (%)', fontsize=13,
fontweight='bold')
ax2.set_ylabel('Ocupação (%)', fontsize=11)
ax2.set_xlabel('Tipo de Veículo', fontsize=11)
ax2.legend(title='Empresa', fontsize=10)
ax2.grid(axis='y', alpha=0.3)
ax2.set_ylim(0, 120)
ax2.set_xticklabels(ax2.get_xticklabels(), rotation=45)

plt.tight_layout()
plt.show()
```




```
# 2.3 Análise de Gargalo: Weigh-out vs Cube-out
# Preparando dados por rota para análise detalhada
df_gargalo = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('tipo_veiculo').first().alias('tipo_veiculo'),
    pl.col('peso_kg_total_rota').first().alias('peso_rota'),
```

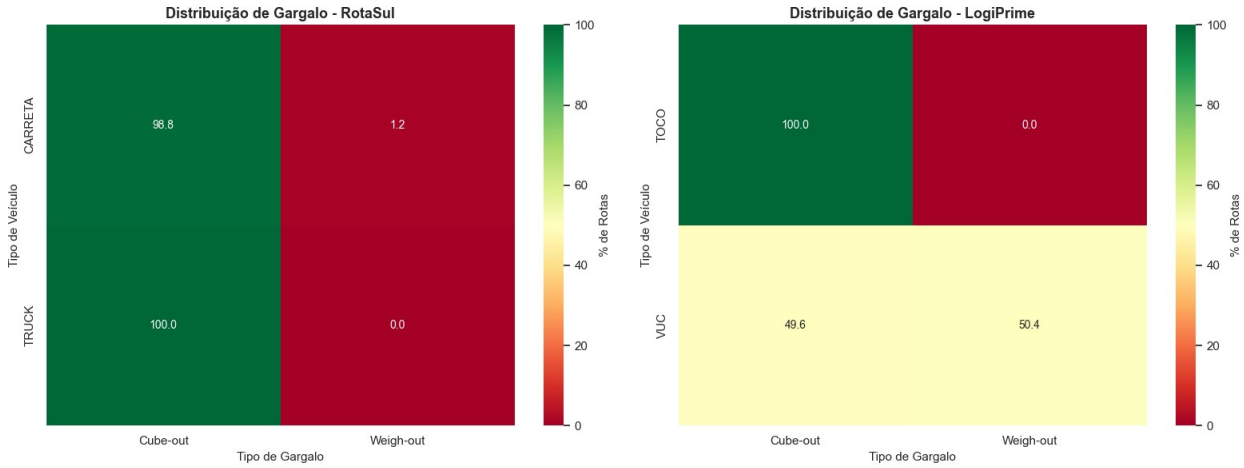
```
    pl.col('volume_m3_total_rota').first().alias('volume_rota')
])

# 2. Agora, adicione as colunas derivadas no DataFrame
df_gargalo = df_gargalo.with_columns([
    pl.when(pl.col('tipo_veiculo') == 'VUC').then(3000)
        .when(pl.col('tipo_veiculo') == 'VAN').then(1500)
        .when(pl.col('tipo_veiculo') == 'TOCO').then(8000)
        .when(pl.col('tipo_veiculo') == 'TRUCK').then(14000)
        .when(pl.col('tipo_veiculo') == 'CARRETA').then(28000)
        .otherwise(15000).alias('cap_peso'),
    pl.when(pl.col('tipo_veiculo') == 'VUC').then(20)
        .when(pl.col('tipo_veiculo') == 'VAN').then(12)
        .when(pl.col('tipo_veiculo') == 'TOCO').then(40)
        .when(pl.col('tipo_veiculo') == 'TRUCK').then(60)
        .when(pl.col('tipo_veiculo') == 'CARRETA').then(100)
        .otherwise(60).alias('cap_volume')
])

df_gargalo = df_gargalo.with_columns([
    (pl.col('peso_rota') / pl.col('cap_peso')).alias('ocup_peso_pct'),
    (pl.col('volume_rota') /
pl.col('cap_volume')).alias('ocup_volume_pct')
])

# Usar pl.lit() para valores literais ou simplificar
df_gargalo = df_gargalo.with_columns([
    pl.when(pl.col('ocup_peso_pct') > pl.col('ocup_volume_pct'))
    .then(pl.lit('Weigh-out'))  # Usar pl.lit() para string literal
    .otherwise(pl.lit('Cube-out'))  # Usar pl.lit() para string
literal
    .alias('gargalo')
])

# Converter para pandas para visualização (se necessário)
gargalo_df = df_gargalo.to_pandas()

# Estatísticas por empresa
print("\n ANÁLISE DE GARGALO: WEIGH-OUT VS CUBE-OUT")
print("="*100)

for empresa in gargalo_df['empresa'].unique():
    empresa_data = gargalo_df[gargalo_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)

    # Contagem de gargalos
    weigh_out = (empresa_data['gargalo'] == 'Weigh-out').sum()
    cube_out = (empresa_data['gargalo'] == 'Cube-out').sum()
```

```
    total = len(empresa_data)

    print(f"  Weigh-out (limitado por peso): {weigh_out:,} rotas
({weigh_out/total*100:.1f}%)")
    print(f"  Cube-out (limitado por volume): {cube_out:,} rotas
({cube_out/total*100:.1f}%)")

    print(f"\n  Detalhe por Tipo de Veículo:")
    print(f"  {'Veículo':<10} {'Weigh-out':<15} {'Cube-out':<15}
{'Preferência':<20}")
    print(f"  {'-'*60}")

    for veiculo in sorted(empresa_data['tipo_veiculo'].unique()):
        veiculo_data = empresa_data[empresa_data['tipo_veiculo'] ==
veiculo]
        wo = (veiculo_data['gargalo'] == 'Weigh-out').sum()
        co = (veiculo_data['gargalo'] == 'Cube-out').sum()
        total_v = len(veiculo_data)
        pref = 'Weigh-out' if wo > co else 'Cube-out'

        print(f"  {veiculo:<10} {wo:>6,d} ({wo/total_v*100:>5.1f}%)
{co:>6,d} ({co/total_v*100:>5.1f}%)  {pref:<20}")
```

```
 ANÁLISE DE GARGALO: WEIGH-OUT VS CUBE-OUT
======================================================================
==============================

RotaSul
----------------------------------------------------------------------
------------------------------
  Weigh-out (limitado por peso): 218 rotas (0.3%)
  Cube-out (limitado por volume): 62,582 rotas (99.7%)

  Detalhe por Tipo de Veículo:
  Veículo    Weigh-out       Cube-out        Preferência
  ------------------------------------------------------------
  CARRETA       218 (  1.2%)  18,622 ( 98.8%)  Cube-out
  TRUCK           0 (  0.0%)  43,960 (100.0%)  Cube-out

LogiPrime
----------------------------------------------------------------------
------------------------------
  Weigh-out (limitado por peso): 28,477 rotas (30.2%)
  Cube-out (limitado por volume): 65,723 rotas (69.8%)

  Detalhe por Tipo de Veículo:
  Veículo    Weigh-out       Cube-out        Preferência
  ------------------------------------------------------------
```

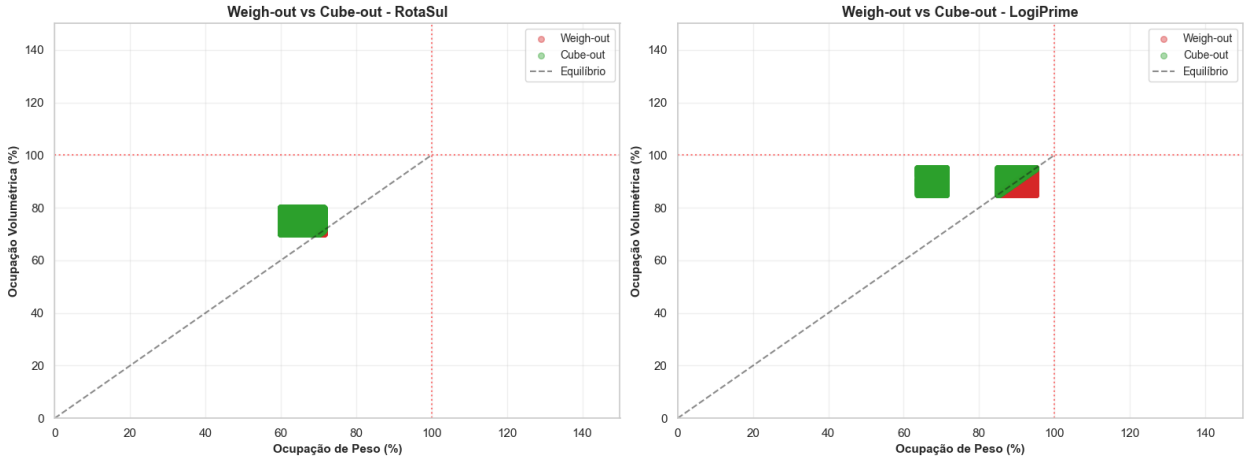
```
  TOCO              0 (   0.0%)  37,680 (100.0%)  Cube-out
  VUC          28,477 ( 50.4%)  28,043 ( 49.6%)  Weigh-out
```

```
# Visualização: Heatmap de gargalos
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for idx, empresa in enumerate(gargalo_df['empresa'].unique()):
    empresa_data = gargalo_df[gargalo_df['empresa'] == empresa]

    # Matriz de gargalos
    gargalo_matrix = pd.crosstab(
        empresa_data['tipo_veiculo'],
        empresa_data['gargalo'],
        normalize='index'
    ) * 100

    ax = axes[idx]
    sns.heatmap(gargalo_matrix, annot=True, fmt='.1f', cmap='RdYlGn',
                cbar_kws={'label': '% de Rotas'}, ax=ax, vmin=0,
vmax=100)
    ax.set_title(f'Distribuição de Gargalo - {empresa}', fontsize=13,
fontweight='bold')
    ax.set_xlabel('Tipo de Gargalo', fontsize=11)
    ax.set_ylabel('Tipo de Veículo', fontsize=11)

plt.tight_layout()
plt.show()
```




```
# Scatter plot: Ocupação de Peso vs Volume
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

for idx, empresa in enumerate(gargalo_df['empresa'].unique()):
    empresa_data = gargalo_df[gargalo_df['empresa'] == empresa]
```

```
    ax = axes[idx]

    # Separando por gargalo
    weigh = empresa_data[empresa_data['gargalo'] == 'Weigh-out']
    cube = empresa_data[empresa_data['gargalo'] == 'Cube-out']

    ax.scatter(weigh['ocup_peso_pct']*100,
weigh['ocup_volume_pct']*100,
               alpha=0.4, s=30, color='#d62728', label='Weigh-out')
    ax.scatter(cube['ocup_peso_pct']*100, cube['ocup_volume_pct']*100,

               alpha=0.4, s=30, color='#2ca02c', label='Cube-out')

    # Linha diagonal de equilíbrio
    ax.plot([0, 100], [0, 100], 'k--', linewidth=1.5, alpha=0.5,
label='Equilíbrio')

    ax.axhline(100, color='red', linestyle=':', alpha=0.5)
    ax.axvline(100, color='red', linestyle=':', alpha=0.5)

    ax.set_xlabel('Ocupação de Peso (%)', fontsize=11,
fontweight='bold')
    ax.set_ylabel('Ocupação Volumétrica (%)', fontsize=11,
fontweight='bold')
    ax.set_title(f'Weigh-out vs Cube-out - {empresa}', fontsize=13,
fontweight='bold')
    ax.set_xlim(0, 150)
    ax.set_ylim(0, 150)
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



- A matriz Weigh-out vs. Cube-out demonstra com mais clareza esse ponto, uma vez que a RotaSul opera quase que integralmente em regime Cube-out (limitada pelo volume): 98,8% das rotas de CARRETA e 100% das rotas de TRUCK atingem o limite volumétrico antes do limite de peso. Isso indica que a RotaSul transporta cargas leves e volumosas, o que pode ser um perfil típico de produtos como bebidas, embalagens e bens de consumo. Na LogiPrime, o cenário é misto, os TOCOs são 100% Cube-out, enquanto os VUCs dividem-se entre Weigh-out (50,4%) e Cube-out (49,6%), revelando uma operação mais heterogênea em termos de densidade de carga.
- Essa informação é decisiva para decisões de M&A sobre renovação de frota, pois comprar veículos maiores em peso (aumento de capacidade em kg) teria impacto mínimo para a RotaSul, já que o gargalo é o volume. A aquisição de veículos com maior capacidade cúbica ou a reconfiguração das baias de carga representaria ganho real. Já para a LogiPrime, qualquer investimento em novos veículos deve considerar a dualidade dos gargalos, especialmente nos VUCs.

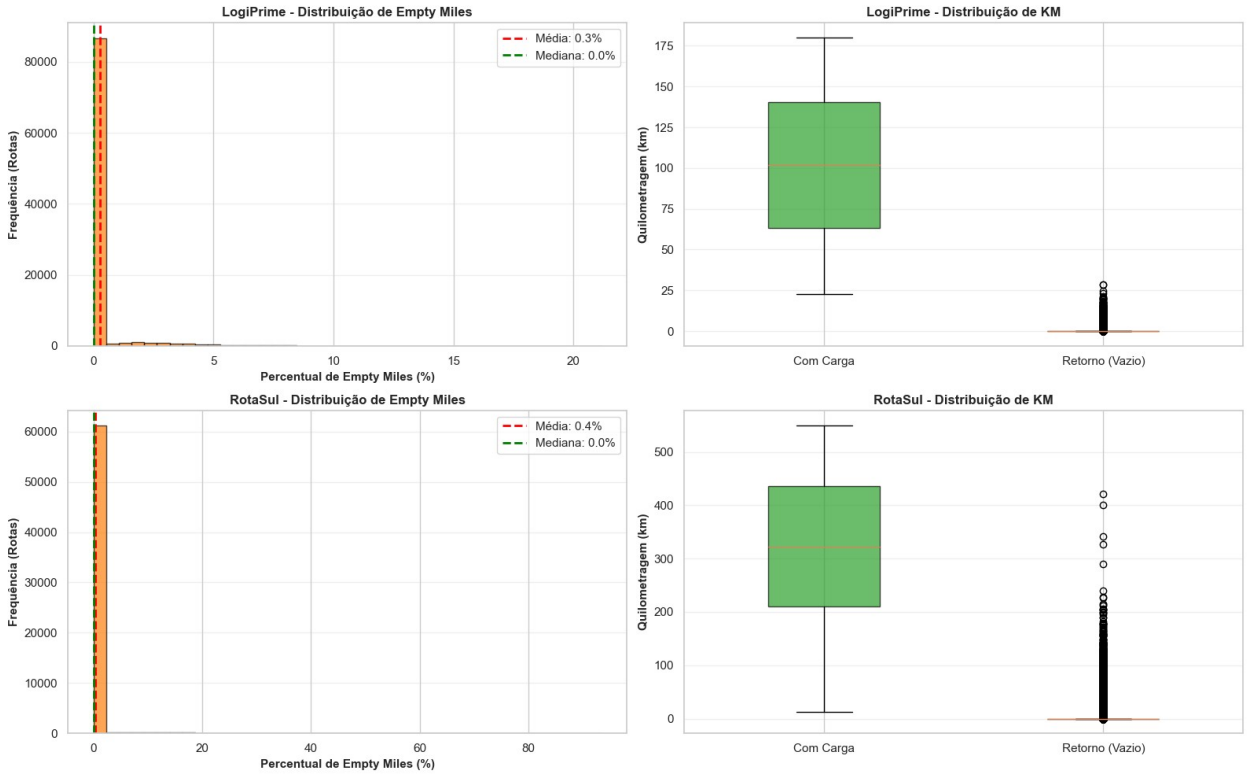
2.4 Custo de Retorno à Base (Empty Miles)

Cálculo de quilômetros rodados com caminhão vazio na volta para o galpão. Qual operação (LogiPrime ou RotaSul) possui o maior percentual de empty miles?

```
# 2.4 Análise de Empty Miles (KM de Retorno à Base)
df_empty_miles = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('km_total_rota').first().alias('km_total'),
    pl.col('km_trecho').sum().alias('km_entregas')
]).with_columns([
    (pl.col('km_total') - pl.col('km_entregas')).alias('km_retorno'),
    ((pl.col('km_total') - pl.col('km_entregas')) / pl.col('km_total')
* 100).alias('pct_retorno')
])

empty_miles_df = df_empty_miles.to_pandas()

# Estatísticas por empresa
print("\n ANÁLISE DE EMPTY MILES (KM DE RETORNO À BASE)")
print("="*100)

for empresa in empty_miles_df['empresa'].unique():
    empresa_data = empty_miles_df[empty_miles_df['empresa'] ==
empresa]

    print(f"\n{empresa}")
    print("-"*100)
    print(f"  Total de rotas: {len(empresa_data):,}")
    print(f"  KM total rodado: {empresa_data['km_total'].sum():,.0f}")
    print(f"  KM com carga: {empresa_data['km_entregas'].sum():,.0f}")
    print(f"  KM de retorno (vazio):
{empresa_data['km_retorno'].sum():,.0f}")
    print(f"\n  Percentual de Empty Miles:")
```

```
    print(f"    Média: {empresa_data['pct_retorno'].mean():.2f}%")
    print(f"    Mediana: {empresa_data['pct_retorno'].median():.2f}%")
    print(f"    Mín: {empresa_data['pct_retorno'].min():.2f}% | Máx:
{empresa_data['pct_retorno'].max():.2f}%")



 ANÁLISE DE EMPTY MILES (KM DE RETORNO À BASE)
======================================================================
==============================

LogiPrime
----------------------------------------------------------------------
------------------------------
  Total de rotas: 94,200
  KM total rodado: 9,643,386
  KM com carga: 9,617,711
  KM de retorno (vazio): 25,674

  Percentual de Empty Miles:
    Média: 0.27%
    Mediana: 0.00%
    Mín: 0.00% | Máx: 21.16%

RotaSul
----------------------------------------------------------------------
------------------------------
  Total de rotas: 62,800
  KM total rodado: 20,367,770
  KM com carga: 20,285,873
  KM de retorno (vazio): 81,897

  Percentual de Empty Miles:
    Média: 0.40%
    Mediana: 0.00%
    Mín: 0.00% | Máx: 93.41%

# Visualização: Distribuição de Empty Miles
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

for idx, empresa in enumerate(empty_miles_df['empresa'].unique()):
    empresa_data = empty_miles_df[empty_miles_df['empresa'] ==
empresa]

    # Histograma de percentual de retorno
    ax1 = axes[idx, 0]
    ax1.hist(empresa_data['pct_retorno'].dropna(), bins=40,
color='#ff7f0e', alpha=0.7, edgecolor='black')
    ax1.axvline(empresa_data['pct_retorno'].mean(), color='red',
linestyle='--', linewidth=2, label=f'Média:
```

```
{empresa_data["pct_retorno"].mean():.1f}%')
    ax1.axvline(empresa_data['pct_retorno'].median(), color='green',
linestyle='--', linewidth=2, label=f'Mediana:
{empresa_data["pct_retorno"].median():.1f}%')
    ax1.set_xlabel('Percentual de Empty Miles (%)', fontsize=11,
fontweight='bold')
    ax1.set_ylabel('Frequência (Rotas)', fontsize=11,
fontweight='bold')
    ax1.set_title(f'{empresa} - Distribuição de Empty Miles',
fontsize=12, fontweight='bold')
    ax1.legend()
    ax1.grid(axis='y', alpha=0.3)

    # Boxplot de KM
    ax2 = axes[idx, 1]
    box_data = [empresa_data['km_entregas'],
empresa_data['km_retorno']]
    bp = ax2.boxplot(box_data, labels=['Com Carga', 'Retorno
(Vazio)'], patch_artist=True, widths=0.4)

    colors = ['#2ca02c', '#d62728']
    for patch, color in zip(bp['boxes'], colors):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)

    ax2.set_ylabel('Quilometragem (km)', fontsize=11,
fontweight='bold')
    ax2.set_title(f'{empresa} - Distribuição de KM', fontsize=12,
fontweight='bold')
    ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```




```python
# Comparação agregada entre empresas
print("\n RESUMO COMPARATIVO: EMPTY MILES POR EMPRESA")
print("="*70)
resumo_empty = empty_miles_df.groupby('empresa').agg({
    'km_total': 'sum',
    'km_entregas': 'sum',
    'km_retorno': 'sum',
    'pct_retorno': ['mean', 'median']
}).round(2)

for empresa in resumo_empty.index:
    total_km = resumo_empty.loc[empresa, ('km_total', 'sum')]
    km_carga = resumo_empty.loc[empresa, ('km_entregas', 'sum')]
    km_ret = resumo_empty.loc[empresa, ('km_retorno', 'sum')]
    pct_media = resumo_empty.loc[empresa, ('pct_retorno', 'mean')]
    pct_mediana = resumo_empty.loc[empresa, ('pct_retorno', 'median')]

    pct_retorno_total = (km_ret / total_km) * 100

    print(f"\n{empresa}")
    print(f"  KM Total: {total_km:,.0f}")
    print(f"  KM com Carga: {km_carga:,.0f}
({km_carga/total_km*100:.1f}%)")
    print(f"  KM Retorno: {km_ret:,.0f} ({pct_retorno_total:.1f}%)")
    print(f"  Média de Empty Miles por Rota: {pct_media:.2f}%")
    print(f"  Mediana de Empty Miles por Rota: {pct_mediana:.2f}%")
```

```
 RESUMO COMPARATIVO: EMPTY MILES POR EMPRESA
======================================================================

LogiPrime
  KM Total: 9,643,386
  KM com Carga: 9,617,711 (99.7%)
  KM Retorno: 25,674 (0.3%)
  Média de Empty Miles por Rota: 0.27%
  Mediana de Empty Miles por Rota: 0.00%

RotaSul
  KM Total: 20,367,770
  KM com Carga: 20,285,873 (99.6%)
  KM Retorno: 81,897 (0.4%)
  Média de Empty Miles por Rota: 0.40%
  Mediana de Empty Miles por Rota: 0.00%
```

- Em relação aos empty miles (quilômetros rodados com caminhão vazio no retorno à base), ambas as operações apresentam índices baixos, LogiPrime com 0,3% e RotaSul com 0,4% do total de quilômetros rodados. A mediana em ambos os casos é 0%, o que indica que a vasta maioria das rotas é circular ou encerra no ponto de entrega sem retorno vazio significativo. Esse dado, embora positivo, merece atenção no nível de rota individual, pois há distribuição de cauda longa nos histogramas, com algumas rotas chegando a 20–80% de empty miles, impactando desproporcionalmente a margem desses percursos específicos.

---

3. CAPILARIDADE E DESENHO DA ROTA

A geometria da entrega difere drasticamente entre operações urbanas (LogiPrime) e regionais (RotaSul). Analisamos extensão, densidade de paradas, dispersão geográfica e "First Mile".

3.1 Extensão Total e Densidade de Paradas

Análise da distribuição de distância total percorrida e número de entregas por rota.

```python
# 3.1 Extensão e Densidade de Rotas
df_capilaridade = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('uf').first().alias('uf'),
    pl.col('km_total_rota').first().alias('km_total'),
    pl.col('num_entregas_rota').first().alias('num_entregas')
]).with_columns([
    (pl.col('km_total') /
pl.col('num_entregas')).alias('km_por_entrega')
])

capil_df = df_capilaridade.to_pandas()
```

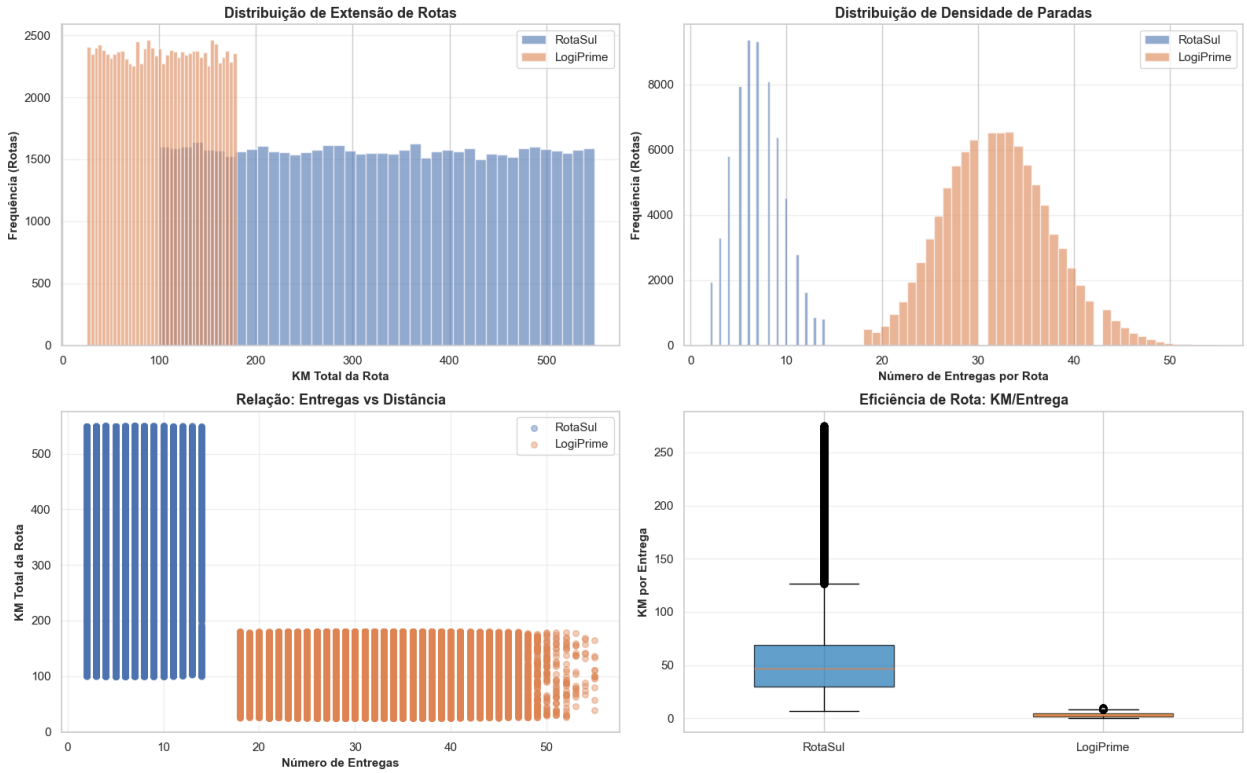
```python
print("\n ANÁLISE DE EXTENSÃO E DENSIDADE DE PARADAS")
print("="*100)

for empresa in capil_df['empresa'].unique():
    empresa_data = capil_df[capil_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)
    print(f"  KM Total por Rota (Média): {empresa_data['km_total'].mean():.1f} km")
    print(f"  KM Total por Rota (Mediana): {empresa_data['km_total'].median():.1f} km")
    print(f"  KM Total por Rota (Min/Max): {empresa_data['km_total'].min():.1f} / {empresa_data['km_total'].max():.1f} km")

    print(f"\n  Entregas por Rota (Média): {empresa_data['num_entregas'].mean():.1f} paradas")
    print(f"  Entregas por Rota (Mediana): {empresa_data['num_entregas'].median():.1f} paradas")
    print(f"  Entregas por Rota (Min/Max): {empresa_data['num_entregas'].min():.0f} / {empresa_data['num_entregas'].max():.0f} paradas")

    print(f"\n  KM por Entrega (Média): {empresa_data['km_por_entrega'].mean():.2f} km/parada")
    print(f"  KM por Entrega (Mediana): {empresa_data['km_por_entrega'].median():.2f} km/parada")
```

```
 ANÁLISE DE EXTENSÃO E DENSIDADE DE PARADAS
====================================================================================================

RotaSul
------------------------------------------------------------------------------------------------------
  KM Total por Rota (Média): 324.3 km
  KM Total por Rota (Mediana): 323.9 km
  KM Total por Rota (Min/Max): 100.0 / 550.0 km

  Entregas por Rota (Média): 7.0 paradas
  Entregas por Rota (Mediana): 7.0 paradas
  Entregas por Rota (Min/Max): 2 / 14 paradas

  KM por Entrega (Média): 55.22 km/parada
  KM por Entrega (Mediana): 46.70 km/parada

LogiPrime
```

```
----------------------------------------------------------------------
------------------------------
  KM Total por Rota (Média): 102.4 km
  KM Total por Rota (Mediana): 102.3 km
  KM Total por Rota (Min/Max): 25.0 / 180.0 km

  Entregas por Rota (Média): 32.0 paradas
  Entregas por Rota (Mediana): 32.0 paradas
  Entregas por Rota (Min/Max): 18 / 55 paradas

  KM por Entrega (Média): 3.30 km/parada
  KM por Entrega (Mediana): 3.20 km/parada
```

```
# Visualização: Comparação de métricas
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. Distribuição de KM Total
ax1 = axes[0, 0]
for empresa in capil_df['empresa'].unique():
    empresa_data = capil_df[capil_df['empresa'] == empresa]
    ax1.hist(empresa_data['km_total'], bins=40, alpha=0.6,
label=empresa)
ax1.set_xlabel('KM Total da Rota', fontsize=11, fontweight='bold')
ax1.set_ylabel('Frequência (Rotas)', fontsize=11, fontweight='bold')
ax1.set_title('Distribuição de Extensão de Rotas', fontsize=13,
fontweight='bold')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# 2. Distribuição de Entregas
ax2 = axes[0, 1]
for empresa in capil_df['empresa'].unique():
    empresa_data = capil_df[capil_df['empresa'] == empresa]
    ax2.hist(empresa_data['num_entregas'], bins=40, alpha=0.6,
label=empresa)
ax2.set_xlabel('Número de Entregas por Rota', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Frequência (Rotas)', fontsize=11, fontweight='bold')
ax2.set_title('Distribuição de Densidade de Paradas', fontsize=13,
fontweight='bold')
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

# 3. Scatter: KM vs Entregas
ax3 = axes[1, 0]
for empresa in capil_df['empresa'].unique():
    empresa_data = capil_df[capil_df['empresa'] == empresa]
    ax3.scatter(empresa_data['num_entregas'],
empresa_data['km_total'],
               alpha=0.4, s=30, label=empresa)
```

```
ax3.set_xlabel('Número de Entregas', fontsize=11, fontweight='bold')
ax3.set_ylabel('KM Total da Rota', fontsize=11, fontweight='bold')
ax3.set_title('Relação: Entregas vs Distância', fontsize=13,
fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# 4. Boxplot de KM por Entrega
ax4 = axes[1, 1]
data_box = [capil_df[capil_df['empresa'] == e]['km_por_entrega'] for e
in capil_df['empresa'].unique()]
bp = ax4.boxplot(data_box, labels=capil_df['empresa'].unique(),
patch_artist=True, widths=0.5)
for patch, color in zip(bp['boxes'], ['#1f77b4', '#ff7f0e']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax4.set_ylabel('KM por Entrega', fontsize=11, fontweight='bold')
ax4.set_title('Eficiência de Rota: KM/Entrega', fontsize=13,
fontweight='bold')
ax4.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



- A geometria das rotas é onde a diferença estrutural entre as duas empresas se torna mais evitende e quantificável. A RotaSul percorre distâncias totais por rota muito

maiores, com distribuição concentrada entre 200 e 500 km por rota, o que indica rotas intermunicipais e interestaduais típicas do Sul do Brasil. A LogiPrime, por sua vez, apresenta rotas significativamente mais curtas, concentradas abaixo de 200 km, reflexo da natureza urbana e perimetral da sua operação em São Paulo e Rio de Janeiro.

- A densidade de paradas aponta para um modelo de serviço radicalmente diferente. A empresa LogiPrime realiza entre 20 e 50 entregas por rota, com distribuição bem dispersa, o que sugere clientes numerosos e próximos entre si. Já a RotaSul apresenta entre 5 e 15 paradas por rota, confirmando que cada viagem atende poucos clientes geograficamente espaçados. Isso se traduz em uma lógica de negócio oposta, pois a LogiPrime maximiza número de clientes por rota (alta frequência, baixo drop size), enquanto a RotaSul maximiza o volume por entrega (baixa frequência, alto drop size).

3.2 First Mile e Dispersão Geográfica

Análise da distância do CD até o primeiro cliente e do padrão de dispersão entre paradas (desvio padrão do km_trecho).

```
# 3.2 First Mile e Dispersão Geográfica
# First Mile: primeira entrega da rota
df_first_mile = df.filter(pl.col('seq_entrega') ==
1).group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('km_trecho').first().alias('km_primeiro_trecho'),
    pl.col('tempo_trecho_min').first().alias('tempo_primeiro_min')
])

# Dispersão: desvio padrão do km_trecho por rota
df_dispersao = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('km_trecho').mean().alias('km_trecho_media'),
    pl.col('km_trecho').std().alias('km_trecho_std'),
    pl.col('km_trecho').count().alias('qtd_trechos')
]).with_columns([
    (pl.col('km_trecho_std') /
pl.col('km_trecho_media')).alias('cv_km_trecho')  # Coef. de Variação
])

# Merge para análise conjunta
df_geom = df_first_mile.join(df_dispersao, on='codigo_rota')

geom_df = df_geom.to_pandas()

print("\n ANÁLISE DE FIRST MILE E DISPERSÃO GEOGRÁFICA")
print("="*100)

for empresa in geom_df['empresa'].unique():
```

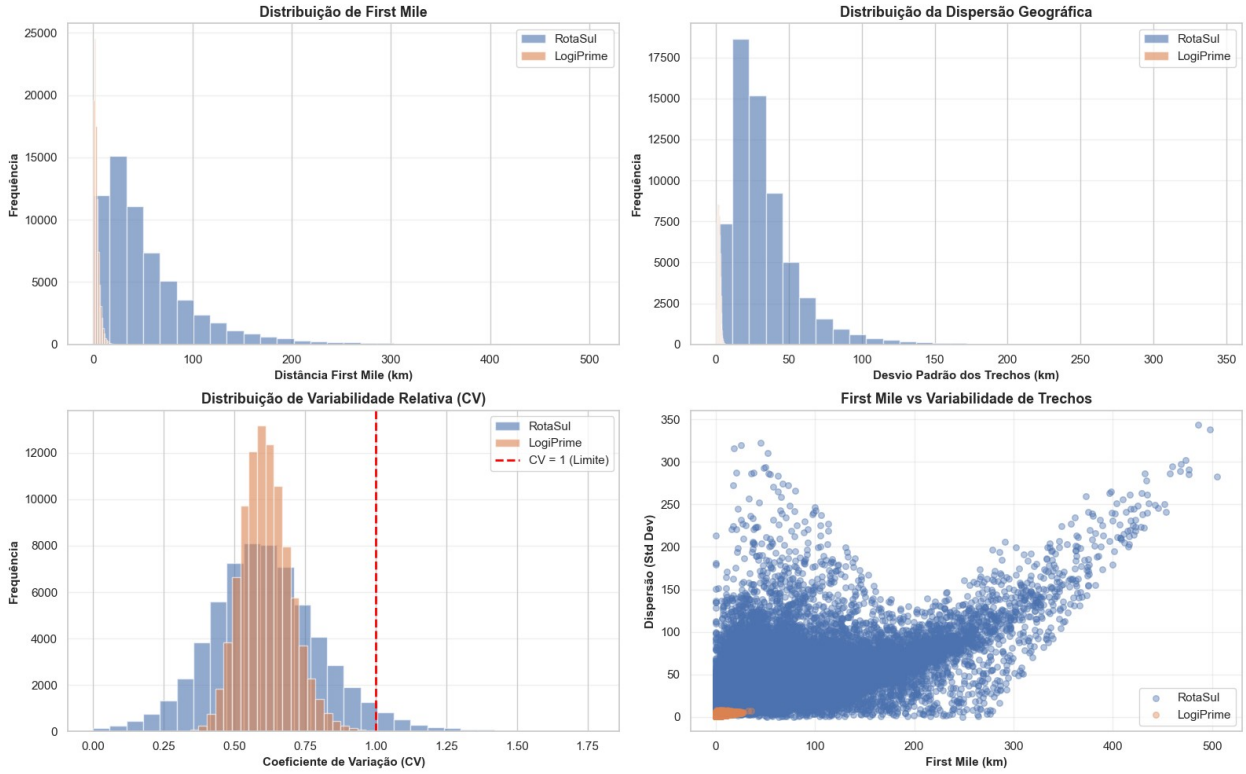
```
    empresa_data = geom_df[geom_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)

    print(f"  FIRST MILE (Distância CD até Primeiro Cliente):")
    print(f"    Média: {empresa_data['km_primeiro_trecho'].mean():.2f}
km")
    print(f"    Mediana:
{empresa_data['km_primeiro_trecho'].median():.2f} km")
    print(f"    Min/Max:
{empresa_data['km_primeiro_trecho'].min():.2f} /
{empresa_data['km_primeiro_trecho'].max():.2f} km")
    print(f"    Tempo Médio:
{empresa_data['tempo_primeiro_min'].mean():.1f} min")

    print(f"\n  DISPERSÃO GEOGRÁFICA (Desvio Padrão dos Trechos):")
    print(f"    Média de KM/Trecho:
{empresa_data['km_trecho_media'].mean():.2f} km")
    print(f"    Std Dev dos Trechos:
{empresa_data['km_trecho_std'].mean():.2f} km")
    print(f"    Coef. de Variação Médio:
{empresa_data['cv_km_trecho'].mean():.3f}")
    print(f"    Interpretação: {'Alta variabilidade' if
empresa_data['cv_km_trecho'].mean() > 1 else 'Baixa variabilidade'}
entre trechos")
```

```
 ANÁLISE DE FIRST MILE E DISPERSÃO GEOGRÁFICA
================================================================================
==============================

RotaSul
----------------------------------------------------------------------------------
------------------------------
  FIRST MILE (Distância CD até Primeiro Cliente):
    Média: 55.01 km
    Mediana: 39.42 km
    Min/Max: 0.00 / 505.04 km
    Tempo Médio: 46.1 min

  DISPERSÃO GEOGRÁFICA (Desvio Padrão dos Trechos):
    Média de KM/Trecho: 55.00 km
    Std Dev dos Trechos: 32.88 km
    Coef. de Variação Médio: 0.612
    Interpretação: Baixa variabilidade entre trechos

LogiPrime
----------------------------------------------------------------------------------
```

```
------------------------------
  FIRST MILE (Distância CD até Primeiro Cliente):
    Média: 3.30 km
    Mediana: 2.53 km
    Min/Max: 0.00 / 35.48 km
    Tempo Médio: 7.5 min

  DISPERSÃO GEOGRÁFICA (Desvio Padrão dos Trechos):
    Média de KM/Trecho: 3.29 km
    Std Dev dos Trechos: 2.02 km
    Coef. de Variação Médio: 0.613
    Interpretação: Baixa variabilidade entre trechos
```

```
# Visualização
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. First Mile por empresa
ax1 = axes[0, 0]
for empresa in geom_df['empresa'].unique():
    empresa_data = geom_df[geom_df['empresa'] == empresa]
    ax1.hist(empresa_data['km_primeiro_trecho'].dropna(), bins=30,
alpha=0.6, label=empresa)
ax1.set_xlabel('Distância First Mile (km)', fontsize=11,
fontweight='bold')
ax1.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax1.set_title('Distribuição de First Mile', fontsize=13,
fontweight='bold')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# 2. Dispersão (Std Dev) por empresa
ax2 = axes[0, 1]
for empresa in geom_df['empresa'].unique():
    empresa_data = geom_df[geom_df['empresa'] == empresa]
    ax2.hist(empresa_data['km_trecho_std'].dropna(), bins=30,
alpha=0.6, label=empresa)
ax2.set_xlabel('Desvio Padrão dos Trechos (km)', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax2.set_title('Distribuição da Dispersão Geográfica', fontsize=13,
fontweight='bold')
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

# 3. Coef. de Variação (CV)
ax3 = axes[1, 0]
for empresa in geom_df['empresa'].unique():
    empresa_data = geom_df[geom_df['empresa'] == empresa]
    ax3.hist(empresa_data['cv_km_trecho'].dropna(), bins=30,
alpha=0.6, label=empresa)
```

```
ax3.set_xlabel('Coeficiente de Variação (CV)', fontsize=11,
fontweight='bold')
ax3.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax3.set_title('Distribuição de Variabilidade Relativa (CV)',
fontsize=13, fontweight='bold')
ax3.axvline(1, color='red', linestyle='--', linewidth=2, label='CV = 1
(Limite)')
ax3.legend()
ax3.grid(axis='y', alpha=0.3)

# 4. Scatter: First Mile vs Dispersão
ax4 = axes[1, 1]
for empresa in geom_df['empresa'].unique():
    empresa_data = geom_df[geom_df['empresa'] == empresa]
    ax4.scatter(empresa_data['km_primeiro_trecho'],
empresa_data['km_trecho_std'],
               alpha=0.4, s=30, label=empresa)
ax4.set_xlabel('First Mile (km)', fontsize=11, fontweight='bold')
ax4.set_ylabel('Dispersão (Std Dev)', fontsize=11, fontweight='bold')
ax4.set_title('First Mile vs Variabilidade de Trechos', fontsize=13,
fontweight='bold')
ax4.legend()
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



- O KM por entrega confirma essa lógica, a RotaSul percorre dezenas de quilômetros entre cada parada, enquanto a LogiPrime conecta clientes vizinhos com trechos curtos. A dispersão geográfica (desvio padrão do km_trecho) da RotaSul é muito elevada, com Coeficiente de Variação próximo a 0,75, indicando que a operação é altamente heterogênea, com algumas rotas que conectam clientes próximos, outras percorrem centenas de quilômetros entre paradas. A LogiPrime apresenta CV próximo a 0,50, operação mais homogênea e previsível, o que facilita o planejamento e a promessa de nível de serviço.
- A análise de First Mile (distância até o primeiro cliente) mostra que a RotaSul tem First Miles longas, frequentemente superiores a 100 km, indicando que os Centros de Distribuição estão relativamente distantes das zonas de entrega, tendo um custo de "saída de galpão" significativo que não agrega valor. A LogiPrime apresenta First Miles mais curtas e compactas, sugerindo CDs melhor posicionados em relação à sua área de cobertura. Na perspectiva de M&A, a localização dos CDs da RotaSul é uma alavanca relevante de otimização, reposicionar ou consolidar instalações poderia reduzir substancialmente o custo do primeiro trecho de cada rota.

---

4. PERFIL DE TEMPOS E RITMO OPERACIONAL

O tempo é o principal driver para cálculo de hora-extra na jornada padrão de 440 minutos. Analisamos matriz de tempo, velocidade operacional, atrito no cliente e estouro de jornada.

4.1 Matriz de Tempo: Distribuição de Atividades

Qual percentual do tempo é gasto dirigindo (km_trecho), esperando no cliente (tempo_espera_pdv_min) e descarregando (tempo_descarga_pdv_min)?

```
# 4.1 Matriz de Tempo
# Agregando tempos por rota
df_tempo = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('tempo_trecho_min').sum().alias('tempo_dirigindo'),
    pl.col('tempo_espera_pdv_min').sum().alias('tempo_espera'),
    pl.col('tempo_descarga_pdv_min').sum().alias('tempo_descarga')
]).with_columns([
    (pl.col('tempo_dirigindo') + pl.col('tempo_espera') +
pl.col('tempo_descarga')).alias('tempo_total')
]).with_columns([
    (pl.col('tempo_dirigindo') / pl.col('tempo_total') *
100).alias('pct_dirigindo'),
    (pl.col('tempo_espera') / pl.col('tempo_total') *
100).alias('pct_espera'),
    (pl.col('tempo_descarga') / pl.col('tempo_total') *
100).alias('pct_descarga')
])
```

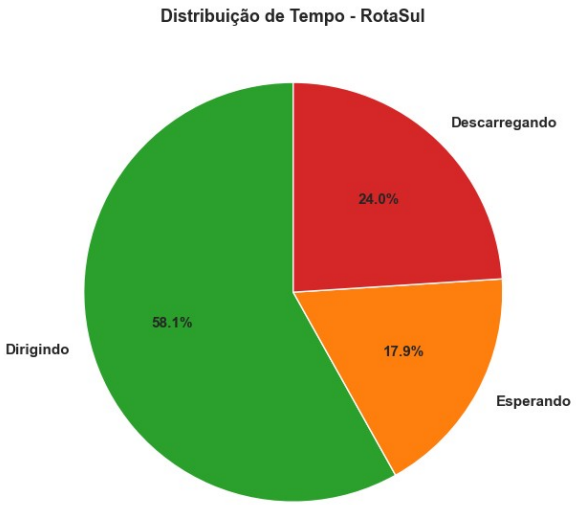
```python
tempo_df = df_tempo.to_pandas()

print("\n MATRIZ DE TEMPO - DISTRIBUIÇÃO DE ATIVIDADES")
print("="*100)

# Resumo por empresa
for empresa in tempo_df['empresa'].unique():
    empresa_data = tempo_df[tempo_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)

    # Valores totais
    tempo_total = empresa_data['tempo_total'].sum()
    tempo_dir = empresa_data['tempo_dirigindo'].sum()
    tempo_esp = empresa_data['tempo_espera'].sum()
    tempo_des = empresa_data['tempo_descarga'].sum()

    print(f"  TEMPO TOTAL ACUMULADO: {tempo_total:,.0f} minutos
({tempo_total/60:,.0f} horas)")
    print(f"\n  Distribuição Agregada:")
    print(f"    Dirigindo: {tempo_dir:,.0f} min
({tempo_dir/tempo_total*100:.1f}%)")
    print(f"    Esperando: {tempo_esp:,.0f} min
({tempo_esp/tempo_total*100:.1f}%)")
    print(f"    Descarregando: {tempo_des:,.0f} min
({tempo_des/tempo_total*100:.1f}%)")

    # Médias por rota
    print(f"\n  Média por Rota:")
    print(f"    Tempo Total: {empresa_data['tempo_total'].mean():.1f}
min")
    print(f"    Dirigindo: {empresa_data['pct_dirigindo'].mean():.1f}
%")
    print(f"    Esperando: {empresa_data['pct_espera'].mean():.1f}%")
    print(f"    Descarregando:
{empresa_data['pct_descarga'].mean():.1f}%")
```

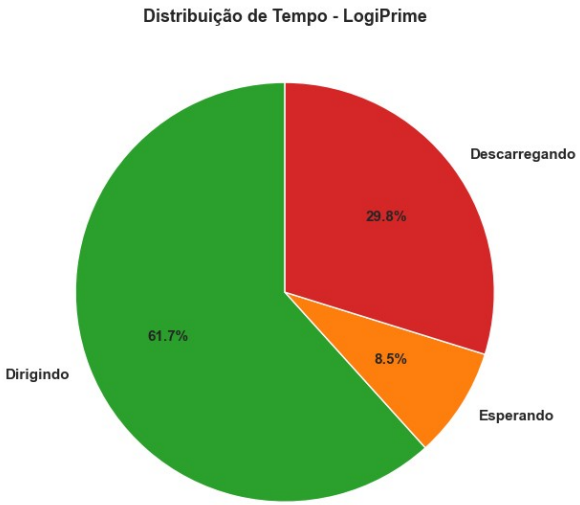
```
 MATRIZ DE TEMPO - DISTRIBUIÇÃO DE ATIVIDADES
====================================================================
==============================

RotaSul
--------------------------------------------------------------------
------------------------------
  TEMPO TOTAL ACUMULADO: 29,282,314 minutos (488,039 horas)
```

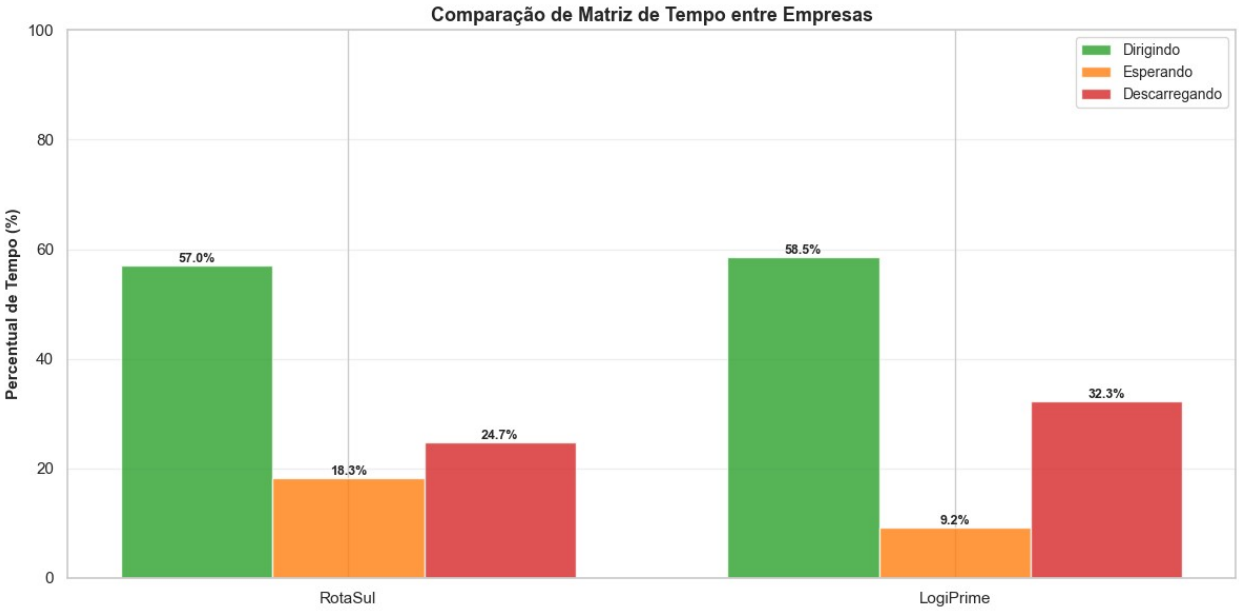
```
  Distribuição Agregada:
    Dirigindo: 17,020,962 min (58.1%)
    Esperando: 5,237,657 min (17.9%)
    Descarregando: 7,023,695 min (24.0%)

  Média por Rota:
    Tempo Total: 466.3 min
    Dirigindo: 57.0%
    Esperando: 18.3%
    Descarregando: 24.7%

LogiPrime
------------------------------------------------------------------------
------------------------------
  TEMPO TOTAL ACUMULADO: 35,445,202 minutos (590,753 horas)

  Distribuição Agregada:
    Dirigindo: 21,869,427 min (61.7%)
    Esperando: 3,016,281 min (8.5%)
    Descarregando: 10,559,493 min (29.8%)

  Média por Rota:
    Tempo Total: 376.3 min
    Dirigindo: 58.5%
    Esperando: 9.2%
    Descarregando: 32.3%
```

```
# Visualização: Gráficos de pizza
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for idx, empresa in enumerate(tempo_df['empresa'].unique()):
    empresa_data = tempo_df[tempo_df['empresa'] == empresa]

    labels = ['Dirigindo', 'Esperando', 'Descarregando']
    sizes = [
        empresa_data['tempo_dirigindo'].sum(),
        empresa_data['tempo_espera'].sum(),
        empresa_data['tempo_descarga'].sum()
    ]
    colors = ['#2ca02c', '#ff7f0e', '#d62728']

    ax = axes[idx]
    wedges, texts, autotexts = ax.pie(sizes, labels=labels,
colors=colors, autopct='%1.1f%%',
                                      startangle=90,
textprops={'fontsize': 11, 'fontweight': 'bold'})
    ax.set_title(f'Distribuição de Tempo - {empresa}', fontsize=13,
fontweight='bold')

plt.tight_layout()
```

```
plt.show()

# Gráfico de barras comparativo
fig, ax = plt.subplots(figsize=(12, 6))

empresas = tempo_df['empresa'].unique()
pct_dirigindo = [tempo_df[tempo_df['empresa'] == e]
['pct_dirigindo'].mean() for e in empresas]
pct_espera = [tempo_df[tempo_df['empresa'] == e]['pct_espera'].mean()
for e in empresas]
pct_descarga = [tempo_df[tempo_df['empresa'] == e]
['pct_descarga'].mean() for e in empresas]

x = np.arange(len(empresas))
width = 0.25

bars1 = ax.bar(x - width, pct_dirigindo, width, label='Dirigindo',
color='#2ca02c', alpha=0.8)
bars2 = ax.bar(x, pct_espera, width, label='Esperando',
color='#ff7f0e', alpha=0.8)
bars3 = ax.bar(x + width, pct_descarga, width, label='Descarregando',
color='#d62728', alpha=0.8)

ax.set_ylabel('Percentual de Tempo (%)', fontsize=11,
fontweight='bold')
ax.set_title('Comparação de Matriz de Tempo entre Empresas',
fontsize=13, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(empresas)
ax.legend(fontsize=10)
ax.grid(axis='y', alpha=0.3)
ax.set_ylim(0, 100)

# Adicionando valores nas barras
for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
                f'{height:.1f}%', ha='center', va='bottom',
fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```







- A matriz de distribuição de tempo dentro da jornada revela como cada empresa "gasta" seu dia operacional e onde estão os principais vetores de ineficiência. A RotaSul aloca 57% do tempo dirigindo, 18,3% esperando nos clientes e 24,7% descarregando. A LogiPrime apresenta 58,5% em deslocamento, apenas 9,2% em espera e 32,3% em descarga. O primeiro dado que chama atenção é o menor tempo de espera da LogiPrime: apenas 9,2% versus 18,3% da RotaSul, resultado direto do sistema de aviso prévio de chegada integrado ao roteirizador da LogiPrime, que agenda o recebimento nos clientes e elimina filas. Esse diferencial é uma das premissas de otimização da fusão (redução de 35% no tempo de espera), e os dados confirmam que o gap real entre as duas operações é substancial.

4.2 Velocidade Operacional e Índice de Estouro de Jornada

Velocidade média de deslocamento e percentual de rotas que extrapolam os 440 minutos de jornada padrão.

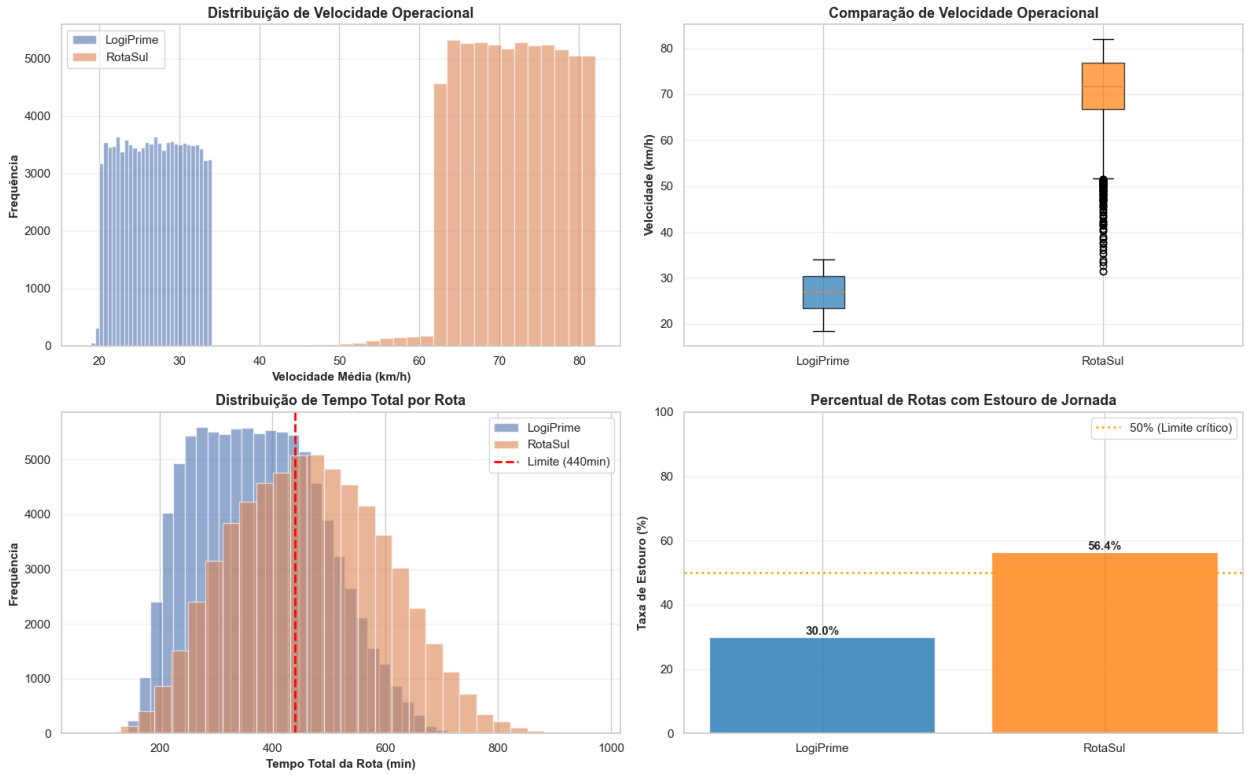
```
# 4.2 Velocidade Operacional e Estouro de Jornada
JORNADA_PADRAO = 440  # minutos

# Velocidade operacional
df_velocidade = df.filter(pl.col('tempo_trecho_min') >
0).with_columns([
    (pl.col('km_trecho') / (pl.col('tempo_trecho_min') /
60)).alias('velocidade_kmh')
]).group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('velocidade_kmh').mean().alias('velocidade_media'),
    pl.col('velocidade_kmh').median().alias('velocidade_mediana')
])

# Estouro de jornada
df_jornada = df.group_by('codigo_rota').agg([
    pl.col('empresa').first().alias('empresa'),
    pl.col('tempo_trecho_min').sum().alias('tempo_dirigindo'),
    pl.col('tempo_espera_pdv_min').sum().alias('tempo_espera'),
    pl.col('tempo_descarga_pdv_min').sum().alias('tempo_descarga')
]).with_columns([
    (pl.col('tempo_dirigindo') + pl.col('tempo_espera') +
pl.col('tempo_descarga')).alias('tempo_total')
]).with_columns([
    pl.when(pl.col('tempo_total') >
JORNADA_PADRAO).then(1).otherwise(0).alias('estouro'),
    (pl.col('tempo_total') - JORNADA_PADRAO).alias('tempo_excedente')
])

jornada_df = df_jornada.to_pandas()
velocidade_df = df_velocidade.to_pandas()

print("\n VELOCIDADE OPERACIONAL E ÍNDICE DE ESTOURO DE JORNADA")
print("="*100)

for empresa in jornada_df['empresa'].unique():
    empresa_vel = velocidade_df[velocidade_df['empresa'] == empresa]
    empresa_jor = jornada_df[jornada_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)

    print(f"  VELOCIDADE OPERACIONAL:")
    print(f"    Média: {empresa_vel['velocidade_media'].mean():.1f}
km/h")
```

```
    print(f"    Mediana:
{empresa_vel['velocidade_mediana'].mean():.1f} km/h")
    print(f"    Min/Max: {empresa_vel['velocidade_media'].min():.1f} /
{empresa_vel['velocidade_media'].max():.1f} km/h")

    total_rotas = len(empresa_jor)
    rotas_estouro = (empresa_jor['estouro'] == 1).sum()
    pct_estouro = (rotas_estouro / total_rotas) * 100

    print(f"\n  ÍNDICE DE ESTOURO DE JORNADA (Limite: {JORNADA_PADRAO}
min):")
    print(f"    Rotas com Estouro: {rotas_estouro:,} / {total_rotas:,}
({pct_estouro:.1f}%)")
    print(f"    Tempo Médio Excedente (rotas com estouro):
{empresa_jor[empresa_jor['estouro']==1]['tempo_excedente'].mean():.1f}
min")
    print(f"    Custo Potencial (hora-extra):
{empresa_jor[empresa_jor['estouro']==1]['tempo_excedente'].sum()/60:.0
f} horas")
```

```
 VELOCIDADE OPERACIONAL E ÍNDICE DE ESTOURO DE JORNADA
================================================================================
==============================

LogiPrime
--------------------------------------------------------------------------------
------------------------------
  VELOCIDADE OPERACIONAL:
    Média: 26.9 km/h
    Mediana: 27.0 km/h
    Min/Max: 18.4 / 34.0 km/h

  ÍNDICE DE ESTOURO DE JORNADA (Limite: 440 min):
    Rotas com Estouro: 28,292 / 94,200 (30.0%)
    Tempo Médio Excedente (rotas com estouro): 72.7 min
    Custo Potencial (hora-extra): 34281 horas

RotaSul
--------------------------------------------------------------------------------
------------------------------
  VELOCIDADE OPERACIONAL:
    Média: 71.7 km/h
    Mediana: 72.0 km/h
    Min/Max: 31.4 / 82.0 km/h

  ÍNDICE DE ESTOURO DE JORNADA (Limite: 440 min):
    Rotas com Estouro: 35,446 / 62,800 (56.4%)
```

```
    Tempo Médio Excedente (rotas com estouro): 122.2 min
    Custo Potencial (hora-extra): 72218 horas

# Visualização
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. Velocidade operacional
ax1 = axes[0, 0]
for empresa in velocidade_df['empresa'].unique():
    empresa_data = velocidade_df[velocidade_df['empresa'] == empresa]
    ax1.hist(empresa_data['velocidade_media'], bins=30, alpha=0.6,
label=empresa)
ax1.set_xlabel('Velocidade Média (km/h)', fontsize=11,
fontweight='bold')
ax1.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax1.set_title('Distribuição de Velocidade Operacional', fontsize=13,
fontweight='bold')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# 2. Comparação de velocidade
ax2 = axes[0, 1]
data_vel = [velocidade_df[velocidade_df['empresa'] == e]
['velocidade_media'] for e in velocidade_df['empresa'].unique()]
bp = ax2.boxplot(data_vel, labels=velocidade_df['empresa'].unique(),
patch_artist=True)
for patch, color in zip(bp['boxes'], ['#1f77b4', '#ff7f0e']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax2.set_ylabel('Velocidade (km/h)', fontsize=11, fontweight='bold')
ax2.set_title('Comparação de Velocidade Operacional', fontsize=13,
fontweight='bold')
ax2.grid(axis='y', alpha=0.3)

# 3. Distribuição de tempo total por rota
ax3 = axes[1, 0]
for empresa in jornada_df['empresa'].unique():
    empresa_data = jornada_df[jornada_df['empresa'] == empresa]
    ax3.hist(empresa_data['tempo_total'], bins=30, alpha=0.6,
label=empresa)
ax3.axvline(JORNADA_PADRAO, color='red', linestyle='--', linewidth=2,
label=f'Limite ({JORNADA_PADRAO}min)')
ax3.set_xlabel('Tempo Total da Rota (min)', fontsize=11,
fontweight='bold')
ax3.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax3.set_title('Distribuição de Tempo Total por Rota', fontsize=13,
fontweight='bold')
ax3.legend()
ax3.grid(axis='y', alpha=0.3)
```

```python
# 4. Taxa de estouro por empresa
ax4 = axes[1, 1]
taxa_estouro = []
empresas = []
for empresa in jornada_df['empresa'].unique():
    empresa_data = jornada_df[jornada_df['empresa'] == empresa]
    taxa = (empresa_data['estouro'].sum() / len(empresa_data)) * 100
    taxa_estouro.append(taxa)
    empresas.append(empresa)

bars = ax4.bar(empresas, taxa_estouro, color=['#1f77b4', '#ff7f0e'],
alpha=0.8)
ax4.axhline(50, color='orange', linestyle=':', linewidth=2, label='50%
(Limite crítico)')
ax4.set_ylabel('Taxa de Estouro (%)', fontsize=11, fontweight='bold')
ax4.set_title('Percentual de Rotas com Estouro de Jornada',
fontsize=13, fontweight='bold')
ax4.set_ylim(0, 100)
ax4.legend()
ax4.grid(axis='y', alpha=0.3)

# Adicionando valores nas barras
for bar in bars:
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
            f'{height:.1f}%', ha='center', va='bottom', fontsize=11,
fontweight='bold')

plt.tight_layout()
plt.show()
```



- A velocidade operacional média reforça a diferença geográfica. LogiPrime opera a aproximadamente 25–30 km/h em média, enquanto a RotaSul circula a 55–70 km/h. Esse dado é esperado e consistente, uma vez que temos um cenário de tráfego urbano denso versus rodovias estaduais e federais com baixa congestionamento. O boxplot de velocidade da RotaSul apresenta alta dispersão, indicando variação considerável entre rotas.

- O índice de estouro de jornada é o dado mais crítico do ponto de vista de risco de Opex. A RotaSul extrapola a jornada padrão de 440 minutos em 56,4% das suas rotas, isso representa mais da metade das operações do dia e que geram obrigação de hora extra. A LogiPrime apresenta 30,0% de estouro, um número alto em termos absolutos, mas significativamente menor. Essa diferença de 26 pontos percentuais traduz-se em custo direto, pois cada rota que extrapola a jornada gera pagamento adicional ao motorista (R$ 36/hora) e ao ajudante (R$ 22/hora), tornando o estouro de jornada da RotaSul uma das maiores oportunidades de redução de Opex identificadas na análise. A combinação do roteirizador com redução de tempo de espera (35%) e redução de tempo de deslocamento (23%) pode mover substancialmente esse indicador para baixo.

---

5. DROP SIZE E PRODUTIVIDADE

Entender o tamanho médio de cada entrega e sua correlação com tempo de descarga. Identificar ineficiências onde veículos grandes são usados para entregas pequenas (ou vice-versa).

5.1 Tamanho de Entrega e Correlação com Tempo de Descarga

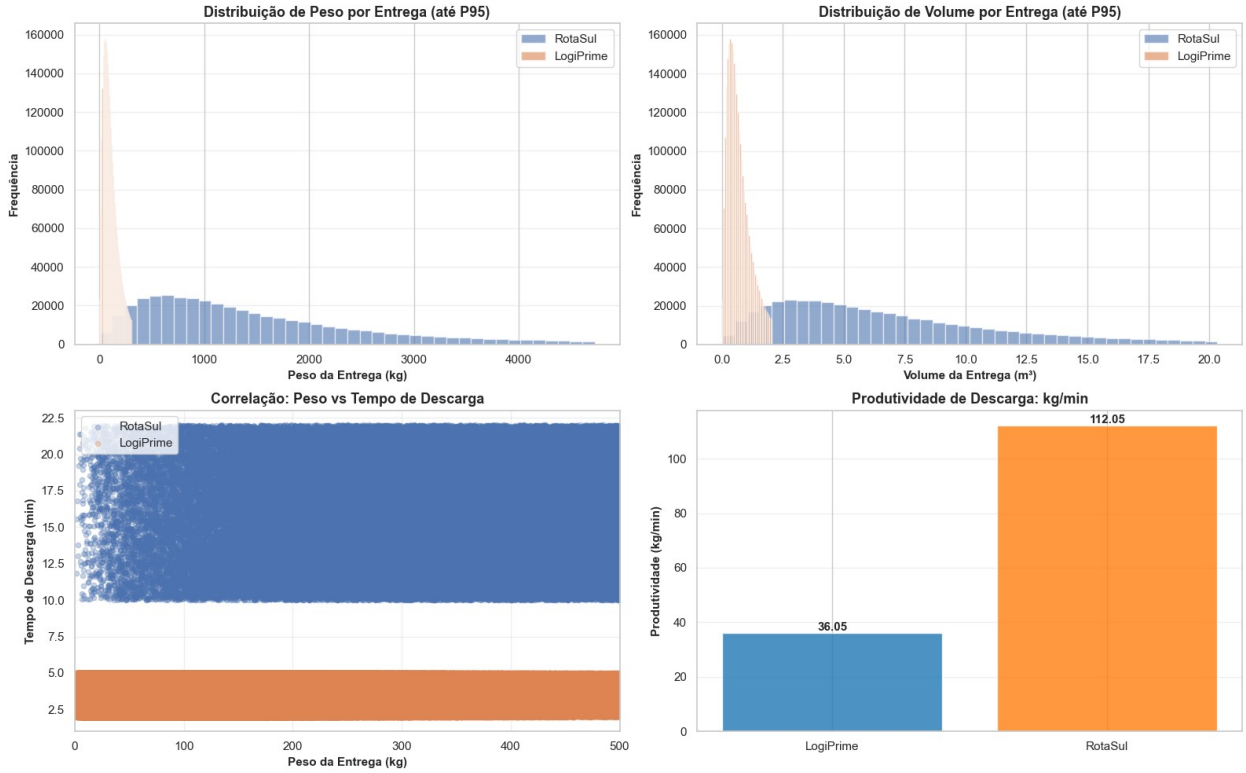
Análise de drop size (peso e volume por entrega) e sua correlação com produtividade de descarga (kg/min ou m³/min).

```
# 5.1 Drop Size e Correlação com Tempo de Descarga
# Filtrando apenas entregas com tempo de descarga > 0 para análise de
produtividade
df_drop_size = df.filter(pl.col('tempo_descarga_pdv_min') >
0).with_columns([
    (pl.col('peso_kg_entrega') /
pl.col('tempo_descarga_pdv_min')).alias('kg_por_min'),
    (pl.col('volume_m3_entrega') /
pl.col('tempo_descarga_pdv_min')).alias('m3_por_min')
])

drop_df = df_drop_size.to_pandas()

# Agregado por empresa
drop_summary = df_drop_size.group_by('empresa').agg([
    pl.col('peso_kg_entrega').mean().alias('peso_medio_entrega'),
    pl.col('peso_kg_entrega').median().alias('peso_mediano_entrega'),
    pl.col('volume_m3_entrega').mean().alias('volume_medio_entrega'),

pl.col('volume_m3_entrega').median().alias('volume_mediano_entrega'),

pl.col('tempo_descarga_pdv_min').mean().alias('tempo_descarga_medio'),
    pl.col('kg_por_min').mean().alias('produtividade_kg_min'),
    pl.col('m3_por_min').mean().alias('produtividade_m3_min')
]).to_pandas()

print("\n ANÁLISE DE DROP SIZE E PRODUTIVIDADE")
print("="*100)

for _, row in drop_summary.iterrows():
    empresa = row['empresa']
    print(f"\n{empresa}")
    print("-"*100)

    print(f"  TAMANHO DE ENTREGA:")
    print(f"    Peso Médio: {row['peso_medio_entrega']:.1f} kg")
    print(f"    Peso Mediano: {row['peso_mediano_entrega']:.1f} kg")
    print(f"    Volume Médio: {row['volume_medio_entrega']:.2f} m³")
    print(f"    Volume Mediano: {row['volume_mediano_entrega']:.2f}
m³")

    print(f"\n  PRODUTIVIDADE DE DESCARGA:")
    print(f"    Tempo Médio de Descarga:
{row['tempo_descarga_medio']:.1f} min")
    print(f"    Produtividade (Peso):
```

```
{row['produtividade_kg_min']:.2f} kg/min")
    print(f"    Produtividade (Volume):
{row['produtividade_m3_min']:.4f} m³/min")



 ANÁLISE DE DROP SIZE E PRODUTIVIDADE
==========================================================================
==============================

LogiPrime
--------------------------------------------------------------------------
------------------------------
  TAMANHO DE ENTREGA:
    Peso Médio: 118.0 kg
    Peso Mediano: 91.8 kg
    Volume Médio: 0.79 m³
    Volume Mediano: 0.61 m³

  PRODUTIVIDADE DE DESCARGA:
    Tempo Médio de Descarga: 3.5 min
    Produtividade (Peso): 36.05 kg/min
    Produtividade (Volume): 0.2403 m³/min

RotaSul
--------------------------------------------------------------------------
------------------------------
  TAMANHO DE ENTREGA:
    Peso Médio: 1706.8 kg
    Peso Mediano: 1254.9 kg
    Volume Médio: 7.73 m³
    Volume Mediano: 5.95 m³

  PRODUTIVIDADE DE DESCARGA:
    Tempo Médio de Descarga: 16.0 min
    Produtividade (Peso): 112.05 kg/min
    Produtividade (Volume): 0.5073 m³/min

# Visualização
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# 1. Distribuição de peso por entrega
ax1 = axes[0, 0]
for empresa in drop_df['empresa'].unique():
    empresa_data = drop_df[drop_df['empresa'] == empresa]
    # Limitando ao P95 para melhor visualização
    p95 = empresa_data['peso_kg_entrega'].quantile(0.95)
    ax1.hist(empresa_data[empresa_data['peso_kg_entrega'] <= p95]
['peso_kg_entrega'],
             bins=40, alpha=0.6, label=empresa)
```

```
ax1.set_xlabel('Peso da Entrega (kg)', fontsize=11, fontweight='bold')
ax1.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax1.set_title('Distribuição de Peso por Entrega (até P95)',
fontsize=13, fontweight='bold')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# 2. Distribuição de volume por entrega
ax2 = axes[0, 1]
for empresa in drop_df['empresa'].unique():
    empresa_data = drop_df[drop_df['empresa'] == empresa]
    p95 = empresa_data['volume_m3_entrega'].quantile(0.95)
    ax2.hist(empresa_data[empresa_data['volume_m3_entrega'] <= p95]
['volume_m3_entrega'],
             bins=40, alpha=0.6, label=empresa)
ax2.set_xlabel('Volume da Entrega (m³)', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Frequência', fontsize=11, fontweight='bold')
ax2.set_title('Distribuição de Volume por Entrega (até P95)',
fontsize=13, fontweight='bold')
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

# 3. Scatter: Peso vs Tempo de Descarga
ax3 = axes[1, 0]
for empresa in drop_df['empresa'].unique():
    empresa_data = drop_df[drop_df['empresa'] == empresa]
    ax3.scatter(empresa_data['peso_kg_entrega'],
empresa_data['tempo_descarga_pdv_min'],
                alpha=0.3, s=20, label=empresa)
ax3.set_xlabel('Peso da Entrega (kg)', fontsize=11, fontweight='bold')
ax3.set_ylabel('Tempo de Descarga (min)', fontsize=11,
fontweight='bold')
ax3.set_title('Correlação: Peso vs Tempo de Descarga', fontsize=13,
fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)
ax3.set_xlim(0, 500)

# 4. Produtividade média por empresa
ax4 = axes[1, 1]
empresas_list = drop_summary['empresa'].tolist()
produtiv_kg = drop_summary['produtividade_kg_min'].tolist()
x_pos = np.arange(len(empresas_list))

bars = ax4.bar(x_pos, produtiv_kg, color=['#1f77b4', '#ff7f0e'],
alpha=0.8)
ax4.set_ylabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax4.set_title('Produtividade de Descarga: kg/min', fontsize=13,
```

```
fontweight='bold')
ax4.set_xticks(x_pos)
ax4.set_xticklabels(empresas_list)
ax4.grid(axis='y', alpha=0.3)

# Adicionando valores nas barras
for bar in bars:
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.2f}', ha='center', va='bottom', fontsize=11,
fontweight='bold')

plt.tight_layout()
plt.show()
```



- O drop size médio confirma a diferença operacional já observada nos pilares anteriores, mas sob a perspectiva do cliente individual. A LogiPrime entrega em média 118 kg por parada, enquanto a RotaSul entrega 1.707 kg por parada, uma diferença de aproximadamente 14 vezes. Isso traduz modelos de negócio completamente distintos, já que a LogiPrime supostamente atende varejistas e pontos de venda menores com reposições frequentes e fracionadas, ao mesmo tempo que a RotaSul abastece supostamente distribuidores, atacadistas ou indústrias com cargas consolidadas e de grande volume.

- Em termos de produtividade de descarga (kg descarregados por minuto), a RotaSul supera a LogiPrime de forma expressiva: 112 kg/min versus 36 kg/min. Esse resultado é estruturalmente coerente, uma vez que descarregar 1.700 kg de uma única nota em um galpão com docas e empilhadeiras é, por unidade de peso, mais rápido do que descarregar 118 kg em 15 caixas no balcão de um mercado por exemplo. A distribuição de percentis revela ainda que a RotaSul tem maior variabilidade (P10 de 22 kg/min versus P90 de 237 kg/min), indicando que a qualidade das instalações dos clientes varia muito, podendo imaginar que alguns são muito bem equipados, já outros sem estrutura de recebimento.

5.2 Análise de Incompatibilidade Veículo vs Drop Size

Identificar padrões onde há desperdício operacional: Carretas fazendo entregas de 50kg ou VUCs com entregas massivas.

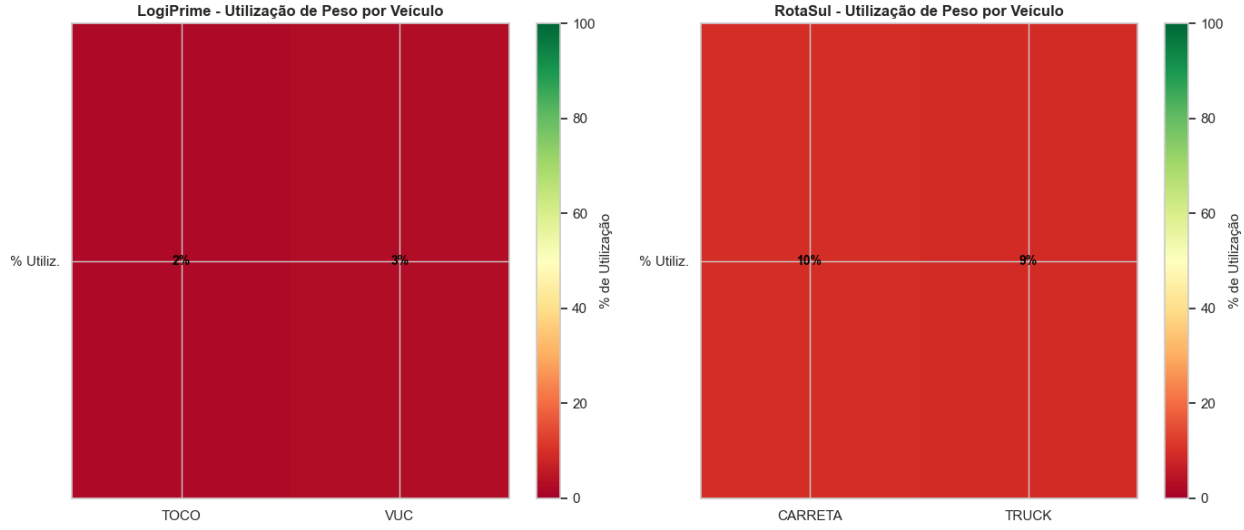
```
# 5.2 Análise de Incompatibilidade Veículo vs Drop Size
# Drop size segmentado por tipo de veículo
df_incompat = df.group_by(['tipo_veiculo', 'empresa']).agg([
    pl.col('peso_kg_entrega').mean().alias('peso_medio'),
    pl.col('volume_m3_entrega').mean().alias('volume_medio'),
    pl.len().alias('qtd_entregas')
])

incompat_df = df_incompat.to_pandas()

# Adicionando capacidades para análise de % de utilização
capacidades_dict = {
    "VUC": {"peso": 3000, "volume": 20},
    "VAN": {"peso": 1500, "volume": 12},
    "TOCO": {"peso": 8000, "volume": 40},
    "TRUCK": {"peso": 14000, "volume": 60},
    "CARRETA": {"peso": 28000, "volume": 100}
}

incompat_df['capacidade_peso'] =
incompat_df['tipo_veiculo'].map(lambda x: capacidades_dict.get(x,
{}).get('peso'))
incompat_df['capacidade_volume'] =
incompat_df['tipo_veiculo'].map(lambda x: capacidades_dict.get(x,
{}).get('volume'))
incompat_df['pct_utiliz_peso'] = (incompat_df['peso_medio'] /
incompat_df['capacidade_peso']) * 100
incompat_df['pct_utiliz_volume'] = (incompat_df['volume_medio'] /
incompat_df['capacidade_volume']) * 100

print("\n ANÁLISE DE INCOMPATIBILIDADE VEÍCULO vs DROP SIZE")
print("="*100)

for empresa in incompat_df['empresa'].unique():
```

```
    empresa_data = incompat_df[incompat_df['empresa'] ==
empresa].sort_values('tipo_veiculo')

    print(f"\n{empresa}")
    print("-"*100)
    print(f"{'Veículo':<10} {'Drop Peso':<12} {'Cap Max':<12} {'%
Util':<10} {'Drop Vol':<12} {'Cap Max':<12} {'% Util':<10}")
    print("-"*100)

    for _, row in empresa_data.iterrows():
        print(f"{row['tipo_veiculo']:<10} {row['peso_medio']:>11.1f}kg
{row['capacidade_peso']:>11.0f}kg {row['pct_utiliz_peso']:>9.1f}%
{row['volume_medio']:>11.2f}m³ {row['capacidade_volume']:>11.1f}m³
{row['pct_utiliz_volume']:>9.1f}%")
```

```
 ANÁLISE DE INCOMPATIBILIDADE VEÍCULO vs DROP SIZE
==========================================================================
==============================

LogiPrime
----------------------------------------------------------------------
------------------------------
Veículo    Drop Peso    Cap Max      % Util     Drop Vol     Cap Max
% Util
----------------------------------------------------------------------
------------------------------
TOCO             168.8kg        8000kg       2.1%        1.13m³
40.0m³       2.8%
VUC               84.3kg        3000kg       2.8%        0.56m³
20.0m³       2.8%

RotaSul
----------------------------------------------------------------------
------------------------------
Veículo    Drop Peso    Cap Max      % Util     Drop Vol     Cap Max
% Util
----------------------------------------------------------------------
------------------------------
CARRETA         2680.8kg       28000kg       9.6%       10.72m³
100.0m³      10.7%
TRUCK           1288.7kg       14000kg       9.2%        6.44m³
60.0m³      10.7%
```

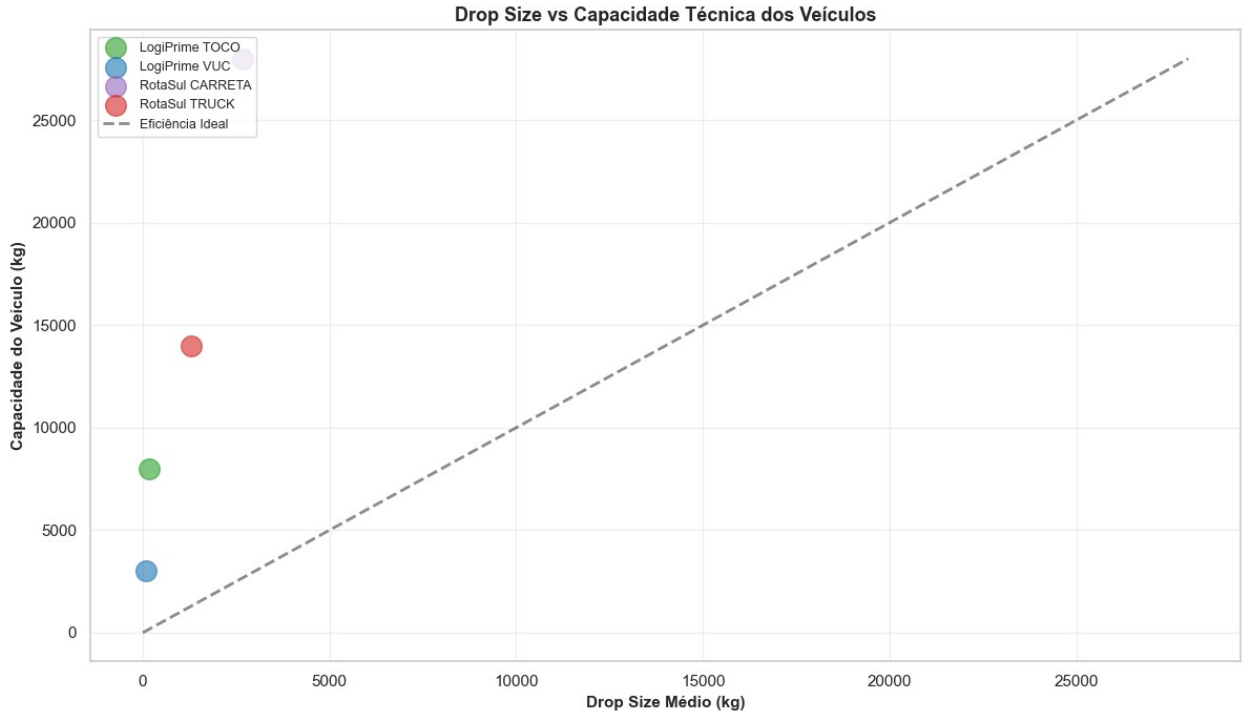
```
# Visualização: Heatmap de incompatibilidade
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```
# Heatmap 1: % Utilização de Peso
for idx, empresa in enumerate(incompat_df['empresa'].unique()):
    empresa_data = incompat_df[incompat_df['empresa'] ==
empresa].set_index('tipo_veiculo')

    ax = axes[idx]

    # Preparando dados para heatmap
    veiculos = sorted(empresa_data.index.tolist())
    pct_peso = empresa_data.loc[veiculos,
'pct_utiliz_peso'].values.reshape(-1, 1)

    im = ax.imshow(pct_peso.T, cmap='RdYlGn', vmin=0, vmax=100,
aspect='auto')

    ax.set_xticks(range(len(veiculos)))
    ax.set_xticklabels(veiculos)
    ax.set_yticks([0])
    ax.set_yticklabels(['% Utiliz.'])
    ax.set_title(f'{empresa} - Utilização de Peso por Veículo',
fontsize=12, fontweight='bold')

    # Adicionando valores nas células
    for i, veiculo in enumerate(veiculos):
        texto = f"{pct_peso[i, 0]:.0f}%"
        ax.text(i, 0, texto, ha='center', va='center', color='white'
if pct_peso[i, 0] > 50 else 'black', fontweight='bold')

    plt.colorbar(im, ax=ax, label='% de Utilização')

plt.tight_layout()
plt.show()
```



```
# Scatter: Drop Size vs Capacidade
fig, ax = plt.subplots(figsize=(12, 7))

for empresa in incompat_df['empresa'].unique():
    empresa_data = incompat_df[incompat_df['empresa'] == empresa]

    colors = {'VUC': '#1f77b4', 'VAN': '#ff7f0e', 'TOCO': '#2ca02c',
'TRUCK': '#d62728', 'CARRETA': '#9467bd'}

    for _, row in empresa_data.iterrows():
        ax.scatter(row['peso_medio'], row['capacidade_peso'],
                   s=200, alpha=0.6, label=f"{empresa}
{row['tipo_veiculo']}",
                   color=colors.get(row['tipo_veiculo'], 'gray'))

# Linha de referência: Drop Size = Capacidade (desperdício zero)
max_val = max(incompat_df['capacidade_peso'].max(),
incompat_df['peso_medio'].max())
ax.plot([0, max_val], [0, max_val], 'k--', linewidth=2,
label='Eficiência Ideal', alpha=0.5)

ax.set_xlabel('Drop Size Médio (kg)', fontsize=11, fontweight='bold')
ax.set_ylabel('Capacidade do Veículo (kg)', fontsize=11,
fontweight='bold')
ax.set_title('Drop Size vs Capacidade Técnica dos Veículos',
fontsize=13, fontweight='bold')
ax.legend(loc='upper left', fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Conclusão: Identificar padrões de desperdício
print("\n PADRÕES IDENTIFICADOS")
print("="*100)

for empresa in incompat_df['empresa'].unique():
    empresa_data = incompat_df[incompat_df['empresa'] == empresa]

    print(f"\n{empresa}")
    print("-"*100)

    # Veículos grandes com drop pequeno
    grandes_com_pequeno = empresa_data[empresa_data['pct_utiliz_peso']
< 20]
    if len(grandes_com_pequeno) > 0:
        print("  DESPERDÍCIO: Veículos grandes com drops pequenos (%
Utilização < 20%):")
        for _, row in grandes_com_pequeno.iterrows():
            print(f"   - {row['tipo_veiculo']}:
```

```
{row['pct_utiliz_peso']:.1f}% de utilização")

    # Veículos pequenos com drop grande
    pequenos_com_grande = empresa_data[empresa_data['pct_utiliz_peso']
> 50]
    if len(pequenos_com_grande) > 0:
        print("\n  RISCO: Veículos pequenos com drops altos (%
Utilização > 50%):")
        for _, row in pequenos_com_grande.iterrows():
            print(f"   - {row['tipo_veiculo']}:
{row['pct_utiliz_peso']:.1f}% de utilização (risco de sobrecarga)")

    # Veículos bem utilizados
    bem_util = empresa_data[(empresa_data['pct_utiliz_peso'] >= 20) &
(empresa_data['pct_utiliz_peso'] <= 80)]
    if len(bem_util) > 0:
        print("\n EFICIENTE: Veículos bem utilizados (20% - 80%):")
        for _, row in bem_util.iterrows():
            print(f"   - {row['tipo_veiculo']}:
{row['pct_utiliz_peso']:.1f}% de utilização")
```




```
 PADRÕES IDENTIFICADOS
================================================================================
==============================

LogiPrime
```

```
------------------------------------------------------------------------
------------------------------
  DESPERDÍCIO: Veículos grandes com drops pequenos (% Utilização <
20%):
   - TOCO: 2.1% de utilização
   - VUC: 2.8% de utilização

RotaSul
------------------------------------------------------------------------
------------------------------
  DESPERDÍCIO: Veículos grandes com drops pequenos (% Utilização <
20%):
   - CARRETA: 9.6% de utilização
   - TRUCK: 9.2% de utilização
```

- A análise de incompatibilidade veículo vs. drop size identificou desperdício operacional em ambas as empresas, mais acentuado na RotaSul, com 9,2% das rotas de TRUCK e 9,6% das CARRETAs utilizam menos de 20% da capacidade de peso para uma única entrega, ou seja, veículos grandes mobilizados para drops pequenos. Na LogiPrime, os índices são menores, sendo 2,1% para TOCO e 2,8% para VUC, refletindo um mix de carga mais adequado à capacidade instalada. Para a RotaSul, esse padrão aponta para uma oportunidade de downsizing seletivo de veículos em determinadas rotas ou a consolidação de cargas de múltiplos clientes próximos em uma única viagem de TRUCK — reduzindo custo por entrega sem perder cobertura geográfica.

```
# 5.3 Velocidade de Descarga (Produtividade Humana)
# Filtrar apenas entregas com tempo de descarga registrado
df_produtividade = df.filter(pl.col('tempo_descarga_pdv_min') >
0).with_columns([
    (pl.col('peso_kg_entrega') /
pl.col('tempo_descarga_pdv_min')).alias('kg_por_min'),
    (pl.col('volume_m3_entrega') /
pl.col('tempo_descarga_pdv_min')).alias('m3_por_min')
]).select(['empresa', 'uf', 'tipo_veiculo', 'peso_kg_entrega',
'volume_m3_entrega',
           'tempo_descarga_pdv_min', 'kg_por_min', 'm3_por_min'])

prod_df = df_produtividade.to_pandas()

print("\n" + "="*120)
print(" ANÁLISE DE VELOCIDADE DE DESCARGA - PRODUTIVIDADE HUMANA")
print("="*120)

#
==========================================================================
======
# PARTE 1: RESUMO POR EMPRESA
#
==========================================================================
======
```

```python
print("\n PRODUTIVIDADE MÉDIA POR EMPRESA")
print("-"*120)

resumo_empresa = df_produtividade.group_by('empresa').agg([
    pl.col('kg_por_min').mean().alias('kg_min_media'),
    pl.col('kg_por_min').median().alias('kg_min_mediana'),
    pl.col('kg_por_min').std().alias('kg_min_std'),
    pl.col('m3_por_min').mean().alias('m3_min_media'),
    pl.col('m3_por_min').median().alias('m3_min_mediana'),
    pl.col('m3_por_min').std().alias('m3_min_std'),
    pl.col('tempo_descarga_pdv_min').mean().alias('tempo_medio'),
    pl.col('peso_kg_entrega').mean().alias('entrega_peso_medio'),
    pl.col('volume_m3_entrega').mean().alias('entrega_volume_medio'),
    pl.len().alias('qtd_entregas')
]).to_pandas()

for _, row in resumo_empresa.iterrows():
    empresa = row['empresa']
    print(f"\n{empresa}")
    print("-"*120)

    print(f"  PRODUTIVIDADE (KG/MIN):")
    print(f"    Média: {row['kg_min_media']:>7.2f} kg/min")
    print(f"    Mediana: {row['kg_min_mediana']:>7.2f} kg/min")
    print(f"    Desvio Padrão: {row['kg_min_std']:>7.2f} kg/min")
    print(f"    Coeficiente de Variação:
{(row['kg_min_std']/row['kg_min_media'])*100:>6.1f}%")

    print(f"\n  PRODUTIVIDADE (M³/MIN):")
    print(f"    Média: {row['m3_min_media']:>7.4f} m³/min")
    print(f"    Mediana: {row['m3_min_mediana']:>7.4f} m³/min")
    print(f"    Desvio Padrão: {row['m3_min_std']:>7.4f} m³/min")

    print(f"\n  CONTEXTO OPERACIONAL:")
    print(f"    Peso médio por entrega:
{row['entrega_peso_medio']:>7.1f} kg")
    print(f"    Volume médio por entrega:
{row['entrega_volume_medio']:>7.2f} m³")
    print(f"    Tempo médio de descarga: {row['tempo_medio']:>7.1f}
min")
    print(f"    Total de entregas analisadas:
{row['qtd_entregas']:>10,.0f}")
```

```
============================================================================
=================================================
 ANÁLISE DE VELOCIDADE DE DESCARGA - PRODUTIVIDADE HUMANA
============================================================================
=================================================
```

```
 PRODUTIVIDADE MÉDIA POR EMPRESA
------------------------------------------------------------------------------------------------------------------------

RotaSul
------------------------------------------------------------------------------------------------------------------------
  PRODUTIVIDADE (KG/MIN):
    Média:  112.05 kg/min
    Mediana:   80.05 kg/min
    Desvio Padrão:  108.43 kg/min
    Coeficiente de Variação:   96.8%

  PRODUTIVIDADE (M³/MIN):
    Média:  0.5073 m³/min
    Mediana:  0.3786 m³/min
    Desvio Padrão:  0.4568 m³/min

  CONTEXTO OPERACIONAL:
    Peso médio por entrega:  1706.8 kg
    Volume médio por entrega:    7.73 m³
    Tempo médio de descarga:    16.0 min
    Total de entregas analisadas:    438,788

LogiPrime
------------------------------------------------------------------------------------------------------------------------
  PRODUTIVIDADE (KG/MIN):
    Média:   36.05 kg/min
    Mediana:   26.91 kg/min
    Desvio Padrão:   31.91 kg/min
    Coeficiente de Variação:   88.5%

  PRODUTIVIDADE (M³/MIN):
    Média:  0.2403 m³/min
    Mediana:  0.1794 m³/min
    Desvio Padrão:  0.2125 m³/min

  CONTEXTO OPERACIONAL:
    Peso médio por entrega:   118.0 kg
    Volume médio por entrega:    0.79 m³
    Tempo médio de descarga:     3.5 min
    Total de entregas analisadas:  3,016,871
```

```
# ANÁLISE POR ESTADO (LogiPrime: SP/RJ vs RotaSul: PR/SC/RS)

print("\n PRODUTIVIDADE POR ESTADO")
print("-"*120)
resumo_estado = df_produtividade.group_by(['empresa', 'uf']).agg([
pl.col('kg_por_min').mean().alias('kg_min_media'),
```

```
pl.col('m3_por_min').mean().alias('m3_min_media'),
pl.col('tempo_descarga_pdv_min').mean().alias('tempo_medio'),
pl.len().alias('qtd_entregas')]).sort(['empresa', 'uf']).to_pandas()

for empresa in resumo_estado['empresa'].unique():
    empresa_data = resumo_estado[resumo_estado['empresa'] == empresa]
    print(f"\n{empresa}")
    print(f"  {'Estado':<10} {'KG/min':<12} {'M³/min':<12} {'Tempo
Médio':<15} {'Qtd Entregas':<15}")
    print(f"  {'-'*65}")

    for _, row in empresa_data.iterrows():
        print(f"  {row['uf']:<10} {row['kg_min_media']:>11.2f}
{row['m3_min_media']:>11.4f} {row['tempo_medio']:>14.1f} min
{row['qtd_entregas']:>14,.0f}")

    # Diferença entre estados
    if len(empresa_data) > 1:
        kg_min_vals = empresa_data['kg_min_media'].values
        diff_pct = ((kg_min_vals.max() - kg_min_vals.min()) /
kg_min_vals.min()) * 100
        print(f"\n  → Variação entre estados: {diff_pct:+.1f}%")
```

```
 PRODUTIVIDADE POR ESTADO
------------------------------------------------------------------------
-----------------------------------------------------

LogiPrime
  Estado     KG/min       M³/min       Tempo Médio     Qtd Entregas
  -----------------------------------------------------------------
  RJ               36.01      0.2399            3.5 min        901,693
  SP               36.07      0.2404            3.5 min      2,115,178

  → Variação entre estados: +0.2%

RotaSul
  Estado     KG/min       M³/min       Tempo Médio     Qtd Entregas
  -----------------------------------------------------------------
  PR              112.33      0.5087           16.0 min        175,767
  RS              111.72      0.5060           16.0 min        130,299
  SC              112.01      0.5069           16.0 min        132,722

  → Variação entre estados: +0.5%
```

```
# ANÁLISE POR TIPO DE VEÍCULO

print("\n PRODUTIVIDADE POR TIPO DE VEÍCULO")
print("-"*120)

resumo_veiculo = df_produtividade.group_by(['empresa',
```

```
'tipo_veiculo']).agg([
    pl.col('kg_por_min').mean().alias('kg_min_media'),
    pl.col('m3_por_min').mean().alias('m3_min_media'),
    pl.col('peso_kg_entrega').mean().alias('peso_medio'),
    pl.col('tempo_descarga_pdv_min').mean().alias('tempo_medio'),
    pl.len().alias('qtd_entregas')
]).sort(['empresa', 'kg_min_media'], descending=[False,
True]).to_pandas()

for empresa in resumo_veiculo['empresa'].unique():
    empresa_data = resumo_veiculo[resumo_veiculo['empresa'] ==
empresa].sort_values('tipo_veiculo')
    print(f"\n{empresa}")
    print(f"  {'Veículo':<10} {'KG/min':<12} {'M³/min':<12} {'Peso
Médio':<14} {'Tempo Médio':<14} {'Entregas':<12}")
    print(f"  {'-'*80}")

    for _, row in empresa_data.iterrows():
        print(f"  {row['tipo_veiculo']:<10}
{row['kg_min_media']:>11.2f} {row['m3_min_media']:>11.4f}
{row['peso_medio']:>13.1f} kg {row['tempo_medio']:>13.1f} min
{row['qtd_entregas']:>11,.0f}")
```

```
 PRODUTIVIDADE POR TIPO DE VEÍCULO
--------------------------------------------------------------------------
-----------------------------------------------------

LogiPrime
  Veículo    KG/min       M³/min       Peso Médio     Tempo Médio
Entregas

--------------------------------------------------------------------------
----------
  TOCO             51.57      0.3437         168.8 kg           3.5
min   1,205,664
  VUC              25.73      0.1715          84.3 kg           3.5
min   1,811,207

RotaSul
  Veículo    KG/min       M³/min       Peso Médio     Tempo Médio
Entregas

--------------------------------------------------------------------------
----------
  CARRETA         175.82      0.7036        2680.8 kg          16.0
min     131,774
  TRUCK            84.69      0.4231        1288.7 kg          16.0
min     307,014
```

```python
# COMPARAÇÃO DIRETA - INSIGHT CHAVE

print("\n INSIGHT COMPARATIVO: QUAL EQUIPE É MAIS PRODUTIVA?")
print("-"*120)

media_logiprime = resumo_empresa[resumo_empresa['empresa'] ==
'LogiPrime']['kg_min_media'].values[0]
media_rotasul = resumo_empresa[resumo_empresa['empresa'] == 'RotaSul']
['kg_min_media'].values[0]

if media_logiprime > media_rotasul:
    diff = ((media_logiprime - media_rotasul) / media_rotasul) * 100
    mais_rapida = 'LogiPrime'
    print(f"\n LogiPrime é {diff:.1f}% MAIS RÁPIDA que RotaSul")
    print(f"  LogiPrime: {media_logiprime:.2f} kg/min")
    print(f"  RotaSul:   {media_rotasul:.2f} kg/min")
else:
    diff = ((media_rotasul - media_logiprime) / media_logiprime) * 100
    mais_rapida = 'RotaSul'
    print(f"\n RotaSul é {diff:.1f}% MAIS RÁPIDA que LogiPrime")
    print(f"  RotaSul:   {media_rotasul:.2f} kg/min")
    print(f"  LogiPrime: {media_logiprime:.2f} kg/min")

print(f"\n INTERPRETAÇÃO:")
if media_logiprime > media_rotasul:
    print(f"   - Equipes de LogiPrime descarregam mais rápido por
unidade de peso")
    print(f"   - Entrégáveis menores (drops menores) = descarga mais
ágil")
    print(f"   - Padrão esperado: operação urbana com muitas pequenas
entregas")
else:
    print(f"   - Equipes de RotaSul descarregam mais rápido por
unidade de peso")
    print(f"   - Entregas maiores organizadas = descarga mais
eficiente")
    print(f"   - Padrão esperado: operação regional com entregas
consolidadas")
```

```
 INSIGHT COMPARATIVO: QUAL EQUIPE É MAIS PRODUTIVA?
------------------------------------------------------------------------
-----------------------------------------------------

 RotaSul é 210.8% MAIS RÁPIDA que LogiPrime
  RotaSul:   112.05 kg/min
  LogiPrime: 36.05 kg/min

 INTERPRETAÇÃO:
   - Equipes de RotaSul descarregam mais rápido por unidade de peso
```

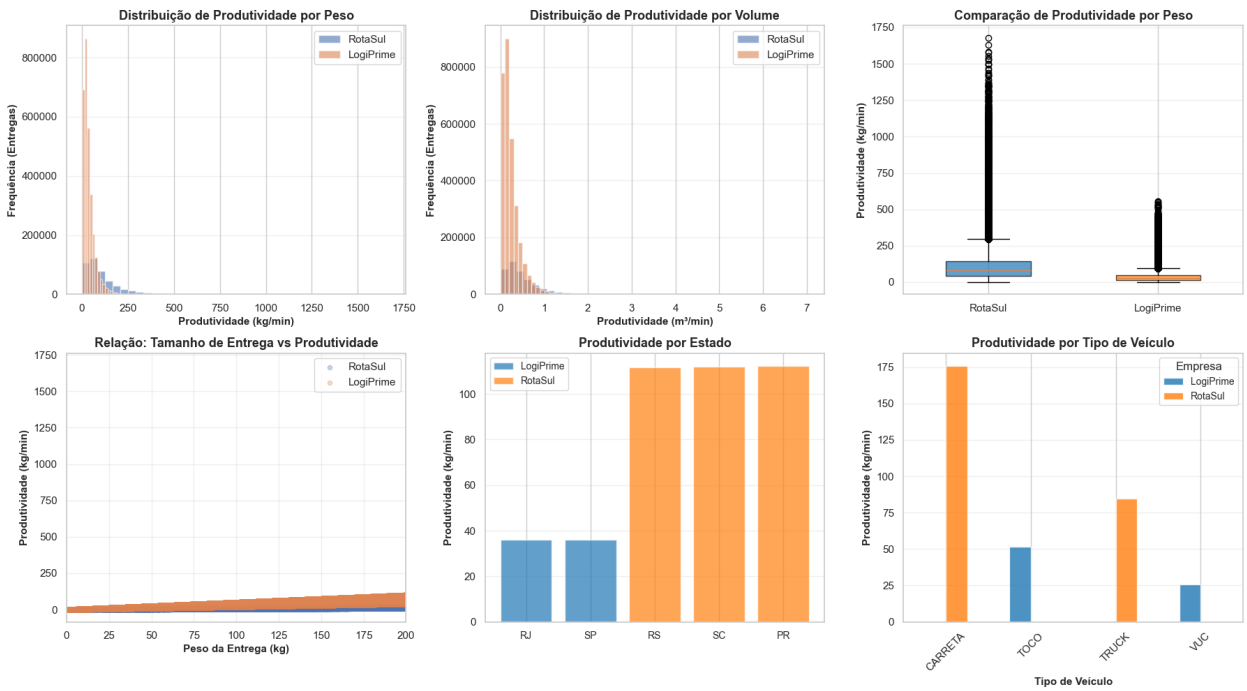
```
   - Entregas maiores organizadas = descarga mais eficiente
   - Padrão esperado: operação regional com entregas consolidadas

# DISTRIBUIÇÃO E OUTLIERS

print("\n\n DISTRIBUIÇÃO DE PRODUTIVIDADE (PERCENTIS)")
print("-"*120)

for empresa in resumo_empresa['empresa'].unique():
    empresa_data = prod_df[prod_df['empresa'] == empresa]
['kg_por_min']

    print(f"\n{empresa}")
    print(f"  P10:  {empresa_data.quantile(0.10):>7.2f} kg/min")
    print(f"  P25:  {empresa_data.quantile(0.25):>7.2f} kg/min")
    print(f"  P50:  {empresa_data.quantile(0.50):>7.2f} kg/min")
    print(f"  P75:  {empresa_data.quantile(0.75):>7.2f} kg/min")
    print(f"  P90:  {empresa_data.quantile(0.90):>7.2f} kg/min")

    iqr = empresa_data.quantile(0.75) - empresa_data.quantile(0.25)
    outliers = len(empresa_data[(empresa_data <
empresa_data.quantile(0.25) - 1.5*iqr) |
                                (empresa_data >
empresa_data.quantile(0.75) + 1.5*iqr)])

    print(f"  Outliers: {outliers:,} entregas
({outliers/len(empresa_data)*100:.1f}%)")

print("\n" + "="*120)
```

```
 DISTRIBUIÇÃO DE PRODUTIVIDADE (PERCENTIS)
------------------------------------------------------------------------
------------------------------------------------------

RotaSul
  P10:    22.73 kg/min
  P25:    42.71 kg/min
  P50:    80.05 kg/min
  P75:   143.57 kg/min
  P90:   237.48 kg/min
  Outliers: 26,600 entregas (6.1%)

LogiPrime
  P10:     8.18 kg/min
  P25:    14.82 kg/min
  P50:    26.91 kg/min
  P75:    46.81 kg/min
  P90:    74.90 kg/min
  Outliers: 164,783 entregas (5.5%)
```

```
==========================================================================
==================================================

# VISUALIZAÇÕES

fig, axes = plt.subplots(2, 3, figsize=(18, 10))

# 1. Distribuição de kg/min por empresa
ax1 = axes[0, 0]
for empresa in prod_df['empresa'].unique():
    empresa_data = prod_df[prod_df['empresa'] == empresa]
    ax1.hist(empresa_data['kg_por_min'].dropna(), bins=40, alpha=0.6,
label=empresa)
ax1.set_xlabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax1.set_ylabel('Frequência (Entregas)', fontsize=11,
fontweight='bold')
ax1.set_title('Distribuição de Produtividade por Peso', fontsize=13,
fontweight='bold')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# 2. Distribuição de m³/min por empresa
ax2 = axes[0, 1]
for empresa in prod_df['empresa'].unique():
    empresa_data = prod_df[prod_df['empresa'] == empresa]
    ax2.hist(empresa_data['m3_por_min'].dropna(), bins=40, alpha=0.6,
label=empresa)
ax2.set_xlabel('Produtividade (m³/min)', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Frequência (Entregas)', fontsize=11,
fontweight='bold')
ax2.set_title('Distribuição de Produtividade por Volume', fontsize=13,
fontweight='bold')
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

# 3. Boxplot comparativo (kg/min)
ax3 = axes[0, 2]
data_kg = [prod_df[prod_df['empresa'] == e]['kg_por_min'].dropna() for
e in prod_df['empresa'].unique()]
bp = ax3.boxplot(data_kg, labels=prod_df['empresa'].unique(),
patch_artist=True, widths=0.5)
for patch, color in zip(bp['boxes'], ['#1f77b4', '#ff7f0e']):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax3.set_ylabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax3.set_title('Comparação de Produtividade por Peso', fontsize=13,
```

```
fontweight='bold')
ax3.grid(axis='y', alpha=0.3)

# 4. Scatter: Peso vs Produtividade
ax4 = axes[1, 0]
for empresa in prod_df['empresa'].unique():
    empresa_data = prod_df[prod_df['empresa'] == empresa]
    ax4.scatter(empresa_data['peso_kg_entrega'],
empresa_data['kg_por_min'],
               alpha=0.3, s=20, label=empresa)
ax4.set_xlabel('Peso da Entrega (kg)', fontsize=11, fontweight='bold')
ax4.set_ylabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax4.set_title('Relação: Tamanho de Entrega vs Produtividade',
fontsize=13, fontweight='bold')
ax4.legend()
ax4.grid(True, alpha=0.3)
ax4.set_xlim(0, 200)

# 5. Comparação por Estado (kg/min)
ax5 = axes[1, 1]
estado_data = resumo_estado.sort_values(['empresa', 'kg_min_media'])
x_pos = np.arange(len(estado_data))
colors_map = {'LogiPrime': '#1f77b4', 'RotaSul': '#ff7f0e'}
colors = [colors_map[e] for e in estado_data['empresa']]
bars = ax5.bar(x_pos, estado_data['kg_min_media'], color=colors,
alpha=0.7)
ax5.set_xticks(x_pos)
ax5.set_xticklabels(estado_data['uf'], fontsize=10)
ax5.set_ylabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax5.set_title('Produtividade por Estado', fontsize=13,
fontweight='bold')
ax5.grid(axis='y', alpha=0.3)

# Legenda customizada
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor='#1f77b4', alpha=0.7,
label='LogiPrime'),
                   Patch(facecolor='#ff7f0e', alpha=0.7,
label='RotaSul')]
ax5.legend(handles=legend_elements, fontsize=10)

# 6. Produtividade por Tipo de Veículo
ax6 = axes[1, 2]
veiculo_pivot = resumo_veiculo.pivot_table(values='kg_min_media',
index='tipo_veiculo',
                                           columns='empresa',
aggfunc='first')
veiculo_pivot.plot(kind='bar', ax=ax6, alpha=0.8, color=['#1f77b4',
```

```
'#ff7f0e'])
ax6.set_ylabel('Produtividade (kg/min)', fontsize=11,
fontweight='bold')
ax6.set_xlabel('Tipo de Veículo', fontsize=11, fontweight='bold')
ax6.set_title('Produtividade por Tipo de Veículo', fontsize=13,
fontweight='bold')
ax6.legend(title='Empresa', fontsize=10)
ax6.grid(axis='y', alpha=0.3)
ax6.set_xticklabels(ax6.get_xticklabels(), rotation=45)

plt.tight_layout()
plt.show()
```




```
# CONCLUSÕES E RECOMENDAÇÕES


print("\n CONCLUSÕES E IMPLICAÇÕES OPERACIONAIS")
print("="*120)

logiprime_data = prod_df[prod_df['empresa'] == 'LogiPrime']
rotasul_data = prod_df[prod_df['empresa'] == 'RotaSul']

# Comparação de variabilidade
cv_logiprime = (logiprime_data['kg_por_min'].std() /
logiprime_data['kg_por_min'].mean()) * 100
cv_rotasul = (rotasul_data['kg_por_min'].std() /
rotasul_data['kg_por_min'].mean()) * 100
```

```python
print(f"\n VARIABILIDADE DE PERFORMANCE:")
if cv_logiprime < cv_rotasul:
    print(f"   LogiPrime é mais CONSISTENTE (CV={cv_logiprime:.1f}% vs {cv_rotasul:.1f}%)")
    print(f"   → Processos padronizados, menor variação entre equipes")
else:
    print(f"   RotaSul é mais CONSISTENTE (CV={cv_rotasul:.1f}% vs {cv_logiprime:.1f}%)")
    print(f"   → Processos padronizados, menor variação entre equipes")

# Implicações de tamanho
print(f"\n IMPACTO DO TAMANHO DE ENTREGA:")
print(f"   LogiPrime: drop médio={logiprime_data['peso_kg_entrega'].mean():.1f}kg → descarga rápida e frequente")
print(f"   RotaSul: drop médio={rotasul_data['peso_kg_entrega'].mean():.1f}kg → descarga menos frequente mas maior")

# Oportunidades
print(f"\n OPORTUNIDADES PÓS-FUSÃO:")
print(f"   • Benchmarking de melhores práticas da operação mais rápida")
print(f"   • Treinamento cruzado de equipes")
print(f"   • Padronização de processos de descarga")
print(f"   • Investimento em equipamentos (jack, dolly) para aumentar produtividade")

print("\n" + "="*120)
```

```
 CONCLUSÕES E IMPLICAÇÕES OPERACIONAIS
========================================================================================================

 VARIABILIDADE DE PERFORMANCE:
   LogiPrime é mais CONSISTENTE (CV=88.5% vs 96.8%)
   → Processos padronizados, menor variação entre equipes

 IMPACTO DO TAMANHO DE ENTREGA:
   LogiPrime: drop médio=118.0kg → descarga rápida e frequente
   RotaSul: drop médio=1706.8kg → descarga menos frequente mas maior

 OPORTUNIDADES PÓS-FUSÃO:
   • Benchmarking de melhores práticas da operação mais rápida
   • Treinamento cruzado de equipes
   • Padronização de processos de descarga
   • Investimento em equipamentos (jack, dolly) para aumentar
```

```
produtividade

==========================================================================
==================================================
```

- A LogiPrime é mais consistente em produtividade (CV de 88,5% versus 96,8% da RotaSul), o que confirma processos de descarga mais padronizados e previsíveis, atributo valioso em operações urbanas com muitas paradas sequenciais onde atrasos se propagam ao longo da rota.

Em síntese, os cinco pilares revelam duas operações que se diferenciam em praticamente todos os eixos mensuráveis, frota, capacidade, geometria de rota, uso do tempo e perfil de carga, mas que são geograficamente complementares e operacionalmente intercambiáveis do ponto de vista de conhecimento técnico e infraestrutura de dados. Essa dualidade é ao mesmo tempo o desafio central da integração e a principal fonte de criação de valor da fusão.